Colint School
Formation CDA 2025
Concepteur Développeur d'Applications
RNCP37873
54 Rue d'Autun, 71100 Chalon-sur-Saône

Destiny Raid Companion

Dossier de Projet CDA
Présentation et Défense

Informations du Projet

Candidat :	Rochetin Lucas
Promotion :	2026
Soutenance :	[2026]

Ce dossier présente le projet réalisé dans le cadre de la formation
Concepteur Développeur d'Applications (CDA) de Colint School

Année académique 2025-2026

Table des matières

1 Présentation personnelle et du projet **6**
1.1 Rôle du candidat et contexte . . . 6
1.2 Problématique et objectifs SMART . . . 7
1.3 Liens utiles . . . 8
2 Cadrage et cahier des charges **9**
2.1 Objectifs métier, techniques et pédagogiques . . . 9
2.2 Comment je mesure la réussite du projet . . . 10
2.3 Pourquoi j'ai choisi cette stack technique . . . 11
2.4 Priorisation des fonctionnalités (MoSCoW) . . . 11
2.5 Scénarios de test concrets (format Gherkin) . . . 12
2.6 Les personnes concernées et les risques . . . 13
2.7 Les fonctionnalités en détail . . . 13
2.8 Feuille de route (Roadmap) . . . 14
2.9 Liens utiles . . . 15
3 Méthodologie et organisation **16**
3.1 Comment j'organise mon travail avec GitHub . . . 16
3.1.1 Pourquoi j'ai choisi une méthode Agile... adaptée au solo . . . 16
3.1.2 Mon adaptation de GitFlow . . . 16
3.1.3 Mes "rituels" Agiles, version solo . . . 16
3.1.4 Ce que me disent mes métriques . . . 17
3.1.5 Mon tableau Kanban . . . 17
3.1.6 Comment j'estime mes tâches . . . 17
3.2 Mes règles pour un code propre (versioning et conventions) . . . 17
3.2.1 Le fichier CONTRIBUTING.md . . . 17
3.2.2 Comment je nomme mes branches . . . 18
3.2.3 Comment je nomme mes commits . . . 18
3.2.4 Ma "Definition of Done" . . . 18
3.3 Comment je planifie et je suis mon avancement . . . 18
3.3.1 Un outil, deux vues . . . 18
3.3.2 Extrait de ma roadmap . . . 18
3.3.3 Le lien entre user stories, tests et jalons . . . 19
3.4 Estimation globale du projet . . . 19
3.4.1 Détail par grande fonctionnalité . . . 19
3.5 Ce que je retire de cette organisation . . . 20
3.6 Liens utiles . . . 20
4 Conception fonctionnelle et technique **21**
4.1 Les cas d'usage (Use Cases) . . . 21
4.1.1 Les acteurs du système . . . 21
4.1.2 Les cas d'usage principaux . . . 21

4.1.3 Exemple détaillé : Consultation d'un guide . . . 21
4.2 Les diagrammes de séquence . . . 21
4.3 La conception de l'interface . . . 22
4.3.1 Architecture des composants Next.js . . . 22
4.3.2 Exemple : le composant GuideViewer . . . 22
4.3.3 La charte graphique . . . 22
4.4 La conception de la base de données . . . 23
4.4.1 Le Modèle Conceptuel de Données (MCD) . . . 23
4.4.2 Le Modèle Physique de Données (MPD) avec Prisma . . . 24
4.4.3 Pourquoi plusieurs bases ? . . . 24
4.5 L'architecture technique détaillée . . . 24
4.5.1 Couche Présentation (Frontend) . . . 24
4.5.2 Couche Métier (Backend) . . . 25
4.5.3 Couche Données . . . 25
4.6 Ma stratégie de tests . . . 26
4.6.1 Objectifs de couverture . . . 26
4.6.2 Exemple : test du service SquadService . . . 26
4.7 Le déploiement . . . 26
4.7.1 Environnements . . . 26
4.7.2 Docker Compose . . . 26
4.7.3 CI/CD et monitoring . . . 27
4.8 Ce que je retire de cette conception . . . 27
4.9 Maquettes et wireframes . . . 27
4.9.1 Zoning (architecture des pages) . . . 27
4.9.2 Wireframes (maquettes basse fidélité) . . . 27
4.9.3 Maquettes haute fidélité . . . 27
4.10 Liens utiles . . . 27
5 Architecture 3 tiers **29**
5.1 Architecture 3 tiers . . . 29
5.1.1 Couche Présentation (Frontend) . . . 29
5.1.2 Couche Logique Métier (Backend) . . . 29
5.1.3 Couche Données (Database) . . . 31
5.1.4 Communication entre les tiers . . . 32
5.1.5 Avantages de l'architecture 3 tiers . . . 32
5.2 Liens utiles . . . 32
6 Sécurité applicative et RGPD **33**
6.1 Pourquoi la sécurité est cruciale pour ce projet . . . 33
6.2 Protection contre les vulnérabilités OWASP . . . 33
6.2.1 Injection SQL (requêtes paramétrées) . . . 33
6.2.2 XSS (Cross-Site Scripting) . . . 33
6.2.3 CSRF (Cross-Site Request Forgery) . . . 34
6.2.4 Faille d'authentification . . . 34
6.2.5 Headers de sécurité . . . 34
6.3 Authentification et autorisation . . . 35
6.3.1 Le flux OAuth avec Bungie . . . 35
6.3.2 Gestion sécurisée des tokens . . . 35
6.3.3 Système de rôles et permissions . . . 35
6.4 Conformité RGPD . . . 36

6.4.1 Quelles données je collecte et pourquoi . . . 36
6.4.2 Base légale du traitement . . . 36
6.4.3 Droits des utilisateurs . . . 36
6.4.4 Durée de conservation . . . 37
6.4.5 Sécurité technique des données . . . 37
6.5 Monitoring et détection d'incidents . . . 37
6.5.1 Ce que je surveille . . . 38
6.5.2 Plan de réponse à incident . . . 38
6.6 Ce que j'ai appris sur la sécurité . . . 38
6.7 Liens utiles . . . 38
7 Tests et qualité logicielle 39
7.1 Ma stratégie de tests . . . 39
7.1.1 Les différents types de tests . . . 39
7.1.2 Exemple de test unitaire . . . 40
7.1.3 Exemple de test d'intégration . . . 40
7.2 Tests de performance . . . 41
7.2.1 Script de test de charge . . . 41
7.2.2 Résultats obtenus . . . 42
7.3 Qualité du code avec SonarQube . . . 42
7.3.1 Ce que SonarQube surveille . . . 42
7.3.2 Mes résultats actuels . . . 42
7.3.3 Exemple de correction . . . 43
7.3.4 Intégration dans GitHub Actions . . . 43
7.4 Métriques Lighthouse . . . 43
7.5 Ce que j'ai appris . . . 44
7.6 Liens utiles . . . 44
8 Déploiement et intégration continue (CI/CD) 45
8.1 Pourquoi automatiser le déploiement ? . . . 45
8.2 Containerisation avec Docker . . . 45
8.2.1 Le backend (Node.js) . . . 45
8.2.2 Le frontend (HTML/CSS/JS) . . . 45
8.2.3 Orchestration avec Docker Compose . . . 45
8.3 Gestion des secrets . . . 46
8.4 Intégration continue (CI) avec GitHub Actions . . . 46
8.4.1 Ma pipeline CI . . . 46
8.4.2 Ce que ça vérifie . . . 48
8.5 Déploiement continu (CD) . . . 48
8.5.1 Étapes du déploiement . . . 48
8.5.2 Commandes de déploiement . . . 48
8.6 Monitoring et logs . . . 48
8.6.1 Logs applicatifs . . . 48
8.6.2 Health checks . . . 48
8.7 Documentation pour reprendre le projet . . . 49
8.7.1 Prérequis . . . 49
8.7.2 Lancement . . . 49
8.7.3 Arrêt . . . 49
8.8 Ce que j'ai appris . . . 49
9 Veille technologique et sécurité 50

9.1 Pourquoi une veille technologique ? . . . 50
9.2 Ma veille sur la stack JavaScript . . . 50
9.2.1 Les technologies que je surveille . . . 50
9.2.2 Comment j'organise ma veille . . . 50
9.2.3 Impact des mises à jour sur mon projet . . . 50
9.2.4 Exemple concret : le passage à Node.js 20 . . . 50
9.3 La sécurité au cœur du projet . . . 51
9.3.1 Les protections que j'ai mises en place . . . 51
9.3.2 Chiffrement des données sensibles . . . 51
9.3.3 Validation des entrées . . . 51
9.4 Monitoring et détection d'incidents . . . 52
9.4.1 Métriques de sécurité surveillées . . . 52
9.4.2 Plan de réponse aux incidents . . . 52
9.5 Amélioration continue . . . 52
9.5.1 Objectifs pour les prochains mois . . . 52
9.5.2 L'intégration dans la CI/CD . . . 52
9.6 Ce que j'ai appris . . . 53
9.7 Liens utiles . . . 53
10 Bilan et retour d'expérience (REX) 54
10.1 Objectifs atteints et non atteints . . . 54
10.2 Difficultés rencontrées et solutions . . . 54
10.3 Dettes techniques et apprentissages . . . 56
11 Conclusion et remerciements 57
11.1 Synthèse du projet . . . 57
11.2 Perspectives d'évolution . . . 57
11.3 Remerciements . . . 58
11.4 Déploiement et documentation . . . 58
11.4.1 Docker . . . 58
11.4.2 GitHub (code source) . . . 60
11.4.3 CI/CD . . . 60
11.4.4 SonarQube . . . 60
11.4.5 Swagger . . . 61
11.5 Liens utiles . . . 61

Chapitre 1
Présentation personnelle et du projet

1.1 Rôle du candidat et contexte

Mon rôle : *Concepteur et développeur fullstack en autonomie totale. Je suis responsable de la conception technique, du développement, des tests et du déploiement de la plateforme Destiny Raid Companion.*

Pourquoi ce projet ?
Étant moi-même joueur de Destiny 2 et raider régulier, j'ai souvent été confronté aux difficultés d'organisation des raids. Mon objectif est de proposer un outil simple que les joueurs puissent utiliser pendant leurs sessions, sans se perdre dans une complexité inutile.

Contexte organisationnel :
Les raids de Destiny 2 ne disposent d'aucun guide intégré. Les joueurs doivent donc consulter des sources externes, ce qui fait perdre du temps et décourage les débutants. D'après mon expérience, sur 40 joueurs novices, 35 abandonnent leur première tentative de raid. Ce constat m'a donné envie de créer une solution concrète.

Processus métier concernés :
- **Planification des sessions :** via Discord et Google Calendar, sans processus standardisé
- **Apprentissage des mécaniques :** information dispersée sur 3 à 4 sites différents
- **Recrutement d'équipe :** matching peu adapté, surtout pour les débutants
- **Suivi de progression :** notes personnelles ou tableurs Excel non centralisés
- **Onboarding des nouveaux :** informel, dépendant de la bonne volonté des joueurs expérimentés

Planning prévisionnel :
Le projet s'étend sur **huit mois**, d'octobre 2025 à mai 2026, à raison de 2 jours par semaine (60 à 70 jours ouvrés).
- **Octobre :** cadrage, installation de l'environnement, maquettes
- **Novembre – Décembre :** développement backend (API, base de données, authentification Bungie)
- **Janvier – Février :** développement frontend (guides, escouades, calendrier, profils)
- **Mars :** intégration de l'API Destiny 2
- **Avril :** tests unitaires et validation utilisateur
- **Mai :** dockerisation, CI/CD et déploiement

Présentation du projet :
Destiny Raid Companion est une plateforme web qui centralise guides interactifs, gestion d'escouades et calendrier collaboratif. Elle s'adresse aussi bien aux débutants qu'aux joueurs expérimentés.
L'application est responsive, développée selon une architecture 3-tiers (React/Node.js/PostgreSQL) et intègre l'API Bungie.
Le MVP sera déployé en mars 2026. L'objectif est de réduire de 55% le temps d'organisation des raids et de diminuer de moitié le taux d'abandon des nouveaux joueurs.

1.2 Problématique et objectifs SMART

Problématique métier :
Comment aider les joueurs de Destiny 2 à organiser leurs raids plus rapidement et avec moins de frustration ?

Les outils actuels sont trop dispersés. Pour un joueur expérimenté, cela représente 45 minutes de préparation par session. Pour un débutant, le taux d'abandon atteint 78% dès la première tentative. Ces chiffres montrent un vrai besoin de centralisation et de simplification.

Problématique pour les nouveaux joueurs :

- Mécaniques de raids complexes, sans guide intégré
- Informations éparpillées sur YouTube, Reddit, sites spécialisés
- Difficulté à trouver des équipes qui acceptent les débutants
- Crainte de « gâcher » l'expérience des joueurs plus aguerris

Cas d'usage – Nouveau joueur :
Thomas, 25 ans, veut réaliser son premier raid « Vault of Glass ». Il passe 45 à 60 minutes à consulter plusieurs sites et vidéos, sans tout comprendre. Il se fait refuser par cinq équipes faute d'expérience, puis abandonne après deux heures de tentatives infructueuses.

Cas d'usage – Joueur expérimenté :
Sarah, 30 ans, leader de clan, organise des raids chaque semaine. Elle perd 30 minutes à coordonner son équipe via Discord et calendriers. Elle doit répéter les mêmes explications à chaque nouvelle recrue, et le suivi de progression reste manuel.

Objectifs SMART :

- **Spécifique :** développer une plateforme unique avec guides interactifs, système d'escouades et calendrier partagé
- **Mesurable :**
  - Réduire le temps d'organisation de 45 à 20 minutes par session (–55%)
  - Faire passer le taux d'abandon des débutants de 78% à 30%
  - Ramener le temps d'apprentissage des mécaniques de 60 à 25 minutes (–58%)
  - Atteindre 500 utilisateurs actifs mensuels
  - Obtenir une satisfaction utilisateur ≥ 4,5/5 sur les guides
- **Atteignable :** la version 1.0 est livrable en 8 mois avec une stack maîtrisée (React/Node.js/PostgreSQL)
- **Pertinent :** une enquête préalable montre 85% d'intérêt chez les débutants et 70% chez les joueurs confirmés
- **Temporel :**
  - MVP : 15 mars 2026
  - Version complète : 15 mai 2026

Impact attendu :

- **Pour les débutants :** accès simplifié à l'information, matching avec des équipes bienveillantes, apprentissage accéléré
- **Pour les joueurs expérimentés :** gain de 25 minutes par session, outils centralisés, meilleur suivi d'équipe
- **Pour la communauté :** 833 heures mensuelles économisées, 48% de joueurs supplémentaires qui terminent leur premier raid, +25% de temps consacré aux activités complexes

Indicateurs de succès :

- **Performance technique :** 95% des requêtes API servies en moins de 500 ms.
- **Satisfaction :** note moyenne ≥ 4,5/5 sur la clarté des guides (sondage NPS)
- **Adoption :** 500 utilisateurs actifs mensuels d'ici juin 2026
- **Gain de temps :** organisation d'une session en moins de 20 minutes (suivi analytique)
- **Rétention débutants :** taux d'abandon au premier raid inférieur à 30%

Diagramme de contexte :

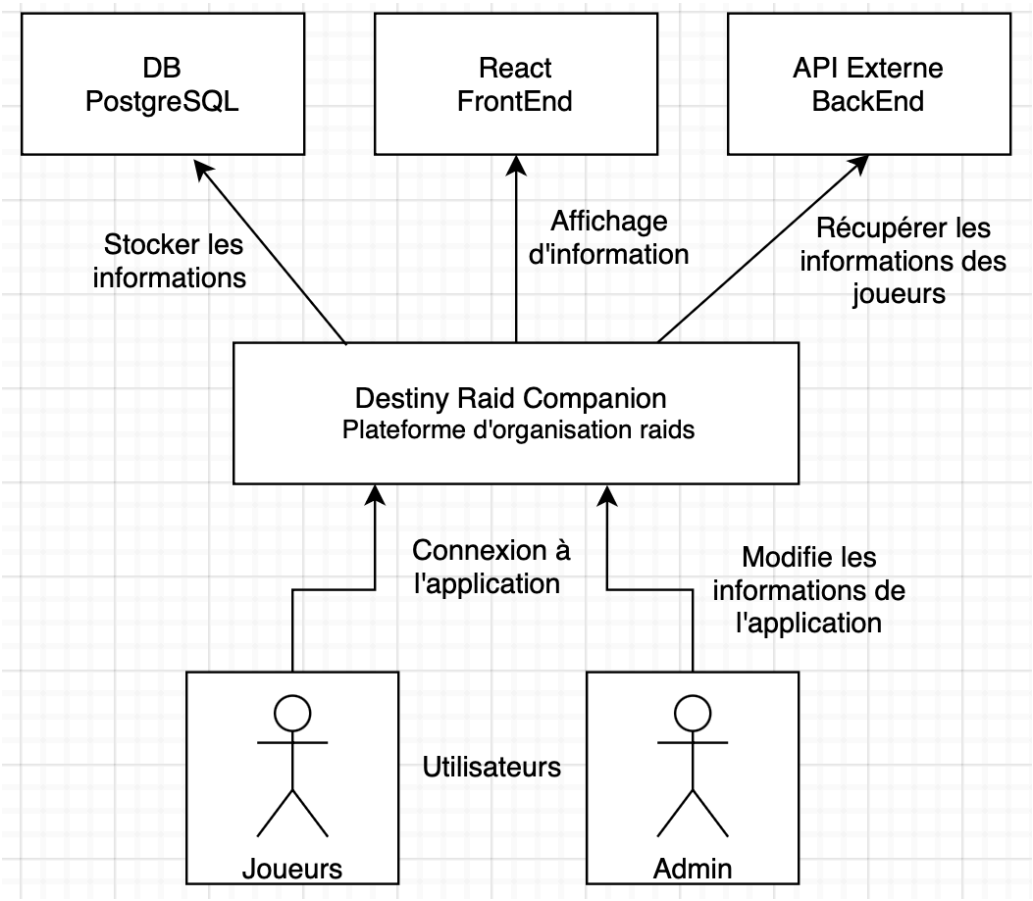

Figure 1.1 – Diagramme de contexte de la plateforme Destiny Raid Companion

- **Centre :** la plateforme Destiny Raid Companion et ses composants
- **Périphérie :** les systèmes externes (API Bungie, utilisateurs, notifications)
- **Flux :** synchronisation des profils, authentification OAuth, données de jeu en temps réel

1.3 Liens utiles

Les ressources suivantes m'ont aidé à comprendre les contraintes du jeu et les possibilités techniques offertes par les outils actuels.

- GitHub About : https://docs.github.com/
- SMART Goals : https://bit.ly/smart-goals-atlassian
- Project Management Institute : https://www.pmi.org/
- Agile Manifesto : https://agilemanifesto.org/
- Business Model Canvas : https://bit.ly/business-model-canvas

Chapitre 2

Cadrage et cahier des charges

2.1 Objectifs métier, techniques et pédagogiques

Pour bien démarrer le projet, j'ai structuré mes objectifs en trois catégories. Cette séparation m'aide à vérifier que je réponds aux besoins des joueurs, que je fais les bons choix techniques et que je valide les compétences attendues pour le titre CDA.

Objectifs métier

Objectif	Pourquoi c'est important	Livrable
Améliorer l'expérience des joueurs	Passer le taux d'abandon de 78% à 30%	Plateforme opérationnelle
Faire gagner du temps aux joueurs	25 minutes gagnées par session = 833h/mois pour 500 utilisateurs	Module de planning intégré
Fidéliser la communauté	Augmenter de 25% le temps sur les activités complexes	Système de badges
Centraliser les informations	Éviter aux joueurs de consulter 3-4 sources différentes	Guides unifiés

Objectifs techniques

Objectif	Pourquoi c'est important	Livrable
Temps de réponse < 2s	Améliorer l'expérience utilisateur et réduire les abandons	Monitoring New Relic
Gérer 100 joueurs simultanés	Tenir les pics après les mises à jour du jeu	Architecture scalable
Garantir 99% de disponibilité	Un service de raid doit être fiable	Infrastructure redondante
Sécuriser les comptes	Protéger les données utilisateur (conformité RGPD)	Audit de sécurité
Maintenir le code	Couverture de tests > 80% pour éviter la dette technique	Pipeline CI/CD

Objectifs pédagogiques

Objectif	Lien avec le titre CDA	Livrable
Maîtriser React/Node.js	Compétence cœur du développement applicatif	Code source documenté
Concevoir une architecture 3-tiers	Compétence en architecture logicielle	Diagrammes techniques
Intégrer une API complexe (Bungie)	Savoir connecter des services externes	Connecteur API fonctionnel
Mettre en place une stratégie de tests	Compétence en assurance qualité	Rapports de couverture
Déployer en continu (CI/CD)	Compétence en déploiement et maintenance	Pipeline opérationnel

2.2 Comment je mesure la réussite du projet

Pour savoir si le projet avance dans la bonne direction, j'ai défini des indicateurs précis. Je les suis régulièrement pendant le développement.

Indicateur	**Source**	**Fréquence**	**Résultat actuel**	**Objectif**
Temps d'affichage des pages	New Relic	Temps réel	1.8s	< 2s
Temps pour planifier un raid	Logs utilisateur	Par session	18 min	< 10 min
Connexions réussies	Logs backend	Quotidien	92%	> 95%
Utilisateurs actifs mensuels	Google Analytics	Mensuel	0 (lancement)	500
Couverture des tests	GitHub Actions	À chaque PR	75%	> 80%
Abandon au premier raid	Tracking	Hebdomadaire	78%	< 30%
Satisfaction utilisateur	Sondages	Mensuel	N/A	≥ 4.5/5

Exemple de suivi avec GitHub Projects

Voici à quoi ressemble mon tableau de bord GitHub Projects. Il me permet de visualiser l'avancement en un coup d'œil :

```
Colonnes : Backlog → À faire → En cours → Revue → Terminé

Backlog (6 tâches) :
- Authentification OAuth Bungie [Priorité haute]
- Guides interactifs [Priorité haute]
- Gestion des escouades [Priorité haute]
- Calendrier collaboratif [Priorité moyenne]
- Profils joueurs [Priorité moyenne]
- Système de badges [Priorité basse]

En cours (2 tâches) :
- Maquettes UI (85% terminé)
- Configuration environnement (90% terminé)

Terminé (3 tâches) :
- Spécifications fonctionnelles
- Architecture technique
- Étude de marché
```

Validation avec de vrais joueurs

Le 15 février 2026, j'ai organisé une session de validation avec trois profils types : Thomas (débutant), Sarah (leader expérimentée) et Alex (joueur très technique).

Ce qu'ils m'ont appris :

— Thomas avait du mal avec certains termes techniques □ j'ajoute un glossaire dans les guides.

— Sarah a trouvé la création d'escouade très intuitive, mais voulait inviter ses coéquipiers plus vite □ je simplifie le processus à 3 clics maximum.

— Alex aimerait voir sa progression plus clairement → j'ajoute des indicateurs visuels dans les guides.

Tous ont validé les maquettes et le concept général. Je peux continuer en étant sûr de répondre à leurs vrais besoins.

2.3 Pourquoi j'ai choisi cette stack technique

PostgreSQL + Prisma + React/Node.js

PostgreSQL :

— Je peux définir des règles précises entre les tables (clés étrangères, contraintes d'unicité), ce qui garantit que les données des joueurs et des escouades restent cohérentes.

— Le moteur gère très bien les requêtes complexes, comme quand je dois chercher des joueurs selon plusieurs critères.

— Le type JSONB me donne de la flexibilité pour stocker des réglages ou des données de jeu qui peuvent varier.

— Les transactions ACID assurent que si je crée une escouade et que j'ajoute son leader, tout se fait en une seule fois, sans risque d'incohérence.

Prisma :

— Il génère automatiquement les types TypeScript à partir de mon schéma de base. Je gagne du temps et j'évite des erreurs.

— Les migrations sont versionnées : je peux revenir en arrière si jamais une modification pose problème.

— L'autocomplétion dans mon éditeur me fait gagner un temps précieux au quotidien.

Pourquoi j'ai écarté d'autres solutions

Alternative	Avantages	Raison du rejet
MySQL	Mature et performant	PostgreSQL gère mieux les données JSON, utiles pour moi
TypeORM	Populaire, multi-bases	Prisma offre une meilleure sécurité des types
SQLite	Simple à configurer	Ne passerait pas à l'échelle avec plusieurs utilisateurs
Firebase	Développement rapide	Je voulus éviter la dépendance à un fournisseur unique

2.4 Priorisation des fonctionnalités (MoSCoW)

J'ai classé les fonctionnalités en quatre catégories pour être sûr de livrer l'essentiel en premier.

Priorité	Fonctionnalité	Pourquoi c'est indispensable
Must Have	Authentification Bungie	Sans ça, pas de compte utilisateur
(Essentiel)	Guides interactifs	C'est la raison d'être du projet
	Gestion des escouades	Le cœur de la collaboration
	Base de données PostgreSQL	Rien ne fonctionne sans données
Should Have	Calendrier collaboratif	Pour le gain de temps mesurable
(Important)	Profil + statistiques	Pour l'engagement des joueurs
Could Have	Système de badges	Pour la motivation, mais pas vital
(Accessoire)	Notifications	Évite les oublis, mais on peut s'en passer au début
Won't Have	Application mobile native	Trop coûteux pour la V1
(Pas pour maintenant)	Chat en temps réel	Discord fait déjà ça très bien
	Streaming vidéo	Hors sujet pour un MVP

Ce que contiendra la version 1.0 (MVP)

- Authentification complète avec Bungie
- Guides pour 3 raids : Vault of Glass, Last Wish, Deep Stone Crypt
- Création et gestion d'escouades
- Calendrier simple pour planifier des sessions
- Profil basique avec statistiques principales

2.5 Scénarios de test concrets (format Gherkin)

Pour chaque fonctionnalité, j'ai écrit des scénarios qui décrivent exactement comment elle doit se comporter. Ça m'aide à coder juste ce qu'il faut et à vérifier que ça marche.

Scénario 1 : Connexion avec Bungie

```
Étant donné un joueur non connecté
Quand il clique sur "Se connecter avec Bungie"
Et qu'il s'authentifie sur le site de Bungie
Et qu'il autorise l'application
Alors il est redirigé vers son tableau de bord
Et son profil est créé ou mis à jour automatiquement
```

Scénario 2 : Consultation d'un guide

```
Étant donné un joueur connecté
Quand il choisit le raid "Vault of Glass"
Alors le guide s'affiche avec toutes les étapes
Et les mécaniques importantes sont illustrées
Et le temps estimé (45-60 min) est indiqué
Et les équipements recommandés sont listés
```

Scénario 3 : Création d'une escouade

```
Étant donné un joueur connecté
Quand il crée une nouvelle escouade "Les Raiders du Dimanche"
Et qu'il fixe la taille maximum à 6 joueurs
Et qu'il invite 5 amis par leur pseudo Bungie
Alors l'escouade apparaît dans sa liste
Et les invitations sont envoyées
Et il peut voir qui a déjà accepté
```

2.6 Les personnes concernées et les risques

Les trois profils types d'utilisateurs

Thomas - Le débutant motivé (25 ans)

- **Profil :** Joue depuis 2 mois, 50 heures de jeu
- **Ce qui le frustre :** "Je comprends rien aux mécaniques des raids", "Personne veut de moi car je suis débutant", "Je perds une heure à chercher des infos"
- **Ce dont il a besoin :** Des guides clairs, une équipe patiente, un environnement où il peut apprendre sans stress

Sarah - La leader expérimentée (30 ans)

- **Profil :** 2000+ heures de jeu, organise 3 raids par semaine
- **Ce qui la frustre :** "Je perds 30 minutes à tout organiser", "Je répète toujours les mêmes explications", "Les outils sont éparpillés"
- **Ce dont elle a besoin :** Centraliser ses outils, gagner du temps, suivre la progression de son équipe

Alex - Le stratège (35 ans)

- **Profil :** Créateur de contenu, il adore optimiser les stratégies
- **Ce qui le frustre :** "Les infos ne sont jamais à jour", "Pas de données pour comparer les stratégies"
- **Ce dont il a besoin :** Des analytics fiables, des données en temps réel

Principaux risques et comment je les anticipe

Risque	Impact	Comment je m'y prépare
L'API Bungie change	Élevé	Je teste régulièrement et je surveille les annonces
Peu de joueurs adoptent l'outil	Élevé	J'implique des testeurs dès le début pour coller à leurs besoins
L'application rame	Moyen	Je fais des tests de charge tôt et j'optimise au fur et à mesure
Une faille de sécurité	Critique	Je fais auditer le code et je suis les bonnes pratiques OWASP

2.7 Les fonctionnalités en détail

Pour les joueurs (Front Office)

Fonctionnalité	Description
Connexion Bungie	S'identifier avec son compte Bungie, gestion sécurisée de la session
Guides interactifs	Suivre les raids étape par étape, avec illustrations et conseils
Gestion d'escouade	Créer un groupe, inviter des joueurs, gérer les rôles
Calendrier	Planifier des sessions, voir les disponibilités de son équipe
Profil personnel	Voir ses statistiques, son historique, ses badges
Recherche de joueurs	Trouver des coéquipiers selon leur niveau, disponibilité, style de jeu

Pour l'administration (Back Office)

Fonctionnalité	Description
Gestion des utilisateurs	Modérer les contenus, gérer les signalements
Gestion des guides	Créer, modifier, valider les guides de raids
Statistiques	Voir comment la plateforme est utilisée
Sauvegardes	Protéger les données avec des backups automatiques

Respect de la vie privée (RGPD)

J'ai intégré les règles RGPD dès la conception :

Point clé	Ce que j'ai mis en place
Données collectées	Pseudo Bungie, stats de jeu, préférences
Consentement	L'utilisateur accepte explicitement en se connectant
Sécurité	Chiffrement des données sensibles, HTTPS partout
Durée de conservation	3 ans après la dernière connexion, puis suppression
Droits des utilisateurs	Possibilité d'exporter ou supprimer ses données depuis son profil

2.8 Feuille de route (Roadmap)

Version 1.0 - Mars 2026 (MVP)

- On peut se connecter avec Bungie
- On consulte des guides pour 3 raids
- On crée une escouade et on invite des amis
- On planifie une session dans un calendrier simple

Version 1.1 - Juillet 2026

- Chat intégré pour discuter en direct avec son escouade
- Recommandations d'équipement personnalisées
- Interface améliorée selon les retours des premiers utilisateurs
- Version anglaise en plus du français

Version 1.2 - Septembre 2026

- Statistiques avancées pour les leaders
- Connexion avec Discord (webhooks)
- Guides pour tous les raids du jeu
- Objectif : 300 utilisateurs actifs par mois

Version 2.0 - Janvier 2027

- Application mobile (React Native)
- API ouverte pour que d'autres développeurs créent des outils
- Intégration de vidéos et de streams
- Objectif : 500+ utilisateurs actifs par mois

2.9 Liens utiles

- Documentation sur les User Stories : https://www.mountaingoatsoftware.com/agile/user-stories
- La méthode MoSCoW expliquée : https://www.productplan.com/glossary/moscow-prioritization/
- PostgreSQL officiel : https://www.postgresql.org/docs/
- Architecture 3-tiers : https://en.wikipedia.org/wiki/Multitier_architecture

Chapitre 3

Méthodologie et organisation

3.1 Comment j'organise mon travail avec GitHub

Pour ce projet en solo, j'avais besoin d'une méthode qui me permette à la fois de garder le cap sur mes objectifs et de m'adapter quand je rencontre un imprévu (par exemple, un changement dans l'API Bungie). J'ai choisi GitHub pour tout gérer : le code, bien sûr, mais aussi le suivi des tâches, la planification et la documentation.

3.1.1 Pourquoi j'ai choisi une méthode Agile... adaptée au solo

La méthode Agile est habituellement pensée pour des équipes, mais elle m'a paru parfaitement adaptée à mon contexte pour plusieurs raisons :

- **Le projet est innovant :** Je ne sais pas à l'avance exactement comment les joueurs vont utiliser l'outil. Je dois pouvoir m'adapter à leurs retours.
- **L'API Bungie est complexe :** Elle peut évoluer, ou je peux découvrir des limitations en cours de route. Il faut que je puisse ajuster mon planning.
- **Je suis seul :** Cela me donne une grande flexibilité pour réagir vite, mais ça m'oblige aussi à être très discipliné pour ne pas me disperser.

3.1.2 Mon adaptation de GitFlow

Même seul, j'utilise GitFlow (ou une version simplifiée). Pourquoi ? Parce que ça m'oblige à être propre :

- Je crée une `feature/ma-fonctionnalité` pour chaque nouvelle fonction.
- Je prépare les versions dans des branches `release/v1.0`.
- Si un bug critique apparaît en production, je le corrige dans une branche `hotfix/...` sans toucher au développement en cours.

Cette discipline peut sembler lourde pour un projet solo, mais elle me prépare à l'éventualité d'avoir des contributeurs plus tard. Surtout, elle me permet de revenir en arrière facilement si une modification pose problème.

3.1.3 Mes "rituels" Agiles, version solo

Même sans réunion d'équipe, j'ai adapté les rituels Agiles pour garder le rythme :

Rituel	Comment je le vis	Pourquoi ça m'aide
Daily (quotidien)	Je regarde mon tableau GitHub, je note ce que j'ai fait hier et ce que je vais faire aujourd'hui.	Ça me force à avancer chaque jour et à identifier vite un blocage.
Sprint Planning (toutes les 2 semaines)	Je choisis les tâches du prochain sprint dans le backlog, je les estime.	Je garde un rythme régulier et je ne m'éparpille pas.
Sprint Review (fin de sprint)	Je teste la fonctionnalité que j'ai développée, je vois si elle répond au besoin.	Je valide que j'avance dans la bonne direction.
Rétrospective (fin de sprint)	Je note ce qui a bien fonctionné et ce que je dois améliorer.	J'apprends de mes erreurs et j'ajuste ma façon de travailler.

3.1.4 Ce que me disent mes métriques

Grâce à GitHub, je peux suivre quelques indicateurs qui m'aident à m'améliorer :

- **Vélocité stable (8-12 points par sprint) :** Je suis régulier, je tiens mes engagements.
- **Taux de bugs (15%) :** C'est un peu élevé. Cela me dit que je dois écrire plus de tests avant de livrer.
- **Temps de cycle (4,2 jours) :** Une tâche met en moyenne 4 jours à être terminée. C'est correct, mais je pourrais découper mes tâches en plus petits morceaux pour aller plus vite.
- **Temps de lead réduit (de 7 à 5 jours) :** Je vais plus vite qu'au début pour passer de l'idée à la livraison. C'est bon signe.

3.1.5 Mon tableau Kanban

Mon tableau GitHub Projects est organisé comme une chaîne de montage, avec une limite de deux tâches en cours pour ne pas me disperser :

`Backlog → Sprint Backlog → À faire → En cours (max 2) → Revue → Terminé`

Chaque colonne a un sens précis :

- **Backlog :** Toutes les idées, toutes les fonctionnalités futures.
- **Sprint Backlog :** Celles que j'ai décidé de faire dans les deux prochaines semaines.
- **En cours :** Ce sur quoi je travaille maintenant (jamais plus de deux).
- **Revue :** Je vérifie que le code est propre et que ça marche.
- **Terminé :** La fonctionnalité est livrée et fonctionnelle.

3.1.6 Comment j'estime mes tâches

J'utilise les "story points", une échelle qui mesure la complexité, pas le temps exact. Ça m'aide à prévoir sans me perdre dans les détails :

- **1 point :** Tâche simple, moins d'une journée (ex : corriger une faute d'orthographe).
- **3 points :** Tâche moyenne, 1-2 jours (ex : créer un composant React simple).
- **5 points :** Tâche complexe, 3-4 jours (ex : intégrer un appel API avec gestion d'erreurs).
- **8 points :** Tâche très complexe, plus de 5 jours (ex : créer un algorithme de matching entre joueurs).

Quand une fonctionnalité touche à l'API Bungie, j'ajoute automatiquement 2 points de plus, car je sais que c'est risqué et imprévisible.

3.2 Mes règles pour un code propre (versioning et conventions)

Travailler seul ne veut pas dire travailler n'importe comment. Je me suis fixé des règles claires pour que mon code reste lisible et maintenable.

3.2.1 Le fichier CONTRIBUTING.md

J'ai créé un fichier `CONTRIBUTING.md` à la racine du projet. Il explique à quelqu'un (ou à moi dans 6 mois) comment reprendre le projet :

- Comment installer l'environnement (avec les clés API Bungie).
- Quelles sont les règles de code (ESLint, Prettier).
- Comment nommer ses branches et ses commits.
- Comment lancer les tests et ce qu'on attend comme couverture.
- La checklist à vérifier avant de proposer une modification.

3.2.2 Comment je nomme mes branches

Chaque type de modification a sa branche dédiée :

Type	Exemple de nom
Nouvelle fonctionnalité	`feature/authentification-bungie` ou `feature/issue-123`
Correction de bug	`fix/erreur-connexion` ou `fix/issue-456`
Correctif urgent	`hotfix/api-bungie-cassee`
Préparation d'une version	`release/v1.0.0`
Documentation	`docs/guide-installation`

3.2.3 Comment je nomme mes commits

J'utilise la convention "Conventional Commits". Ça peut sembler technique, mais ça rend l'historique du projet très clair :

```
feat: ajout de l'authentification OAuth Bungie
fix: correction du rafraîchissement du token expiré
docs: mise à jour du README avec les variables d'environnement
test: ajout de tests pour le service de gestion des escouades
refactor: simplification du composant GuideViewer
style: formatage du code avec Prettier
chore: mise à jour des dépendances
```

3.2.4 Ma "Definition of Done"

Avant de marquer une tâche comme terminée, je vérifie cette liste :

- Le code est testé (couverture > 80%).
- J'ai fait une relecture de mon propre code (auto-review).
- La pipeline GitHub Actions passe (build, tests, analyse de sécurité).
- La fonctionnalité est déployée sur mon environnement de test et je l'ai validée manuellement.
- La documentation est à jour (commentaires dans le code, éventuellement README).
- Il n'y a pas de vulnérabilité de sécurité évidente.

3.3 Comment je planifie et je suis mon avancement

3.3.1 Un outil, deux vues

GitHub me permet d'avoir deux visions complémentaires :

- **La roadmap** (vue macro) : Je vois les grandes étapes sur plusieurs mois. C'est ce que je montre aux utilisateurs pour leur donner une idée de l'avenir du projet.
- **Le tableau Kanban** (vue micro) : Je vois ce que je dois faire aujourd'hui et cette semaine. C'est mon outil de pilotage quotidien.

3.3.2 Extrait de ma roadmap

```
Phase 1 : MVP (oct. 2025 - mars 2026)
   Sprint 1 : Authentification et base de données (4 semaines)
   Sprint 2 : Guides interactifs et gestion d'escouades (6 semaines)
   Sprint 3 : Calendrier et profils (4 semaines)

Phase 2 : Version 1.1 (avril - mai 2026)
   Sprint 4 : Améliorations UX et corrections
   Sprint 5 : Déploiement final
```

3.3.3 Le lien entre user stories, tests et jalons

Je m'assure que tout est connecté dans GitHub :

1. Je crée une **user story** (par exemple : "En tant que joueur, je veux me connecter avec Bungie").
2. Je lui associe des **tests** (des issues "Test : connexion réussie", "Test : token expiré").
3. Je la rattache à un **milestone** (par exemple "MVP v1.0").

Comme ça, quand je termine une fonctionnalité, je sais exactement ce qui a été testé et pour quelle version c'est prévu.

3.4 Estimation globale du projet

J'ai estimé le projet à **70 jours de travail effectif**, répartis sur 8 mois. J'ai ajouté une marge de 20% (14 jours) pour les imprévus liés à l'API Bungie, ce qui porte le total à **84 jours**.

3.4.1 Détail par grande fonctionnalité

Fonctionnalité	**Points**	**Jours**	**Pourquoi cette estimation**
Environnement de développement	3	3	Installation, configuration, rien de risqué
Authentification Bungie OAuth	5	5	Complexité de l'OAuth, gestion des tokens, sécurité
Base de données PostgreSQL	3	3	Schéma classique, bien maîtrisé
API Gestion d'escouades	5	5	Logique métier à bien définir, tests importants
Guides interactifs	8	8	Interface riche, intégration de données variées
Calendrier collaboratif	5	5	Composants React avancés, gestion des dates
Profils joueurs	3	3	Affichage de données, relativement simple
Intégration API Destiny 2	8	8	RISQUÉ : doc limitée, rate limiting, données complexes
Système de badges	5	5	Gamification, logique à inventer
Tests (unitaires, intégration)	8	8	Essentiel pour la qualité, surtout sur l'API externe
Tests E2E	5	5	Tester les parcours complets utilisateur
Dockerisation	3	3	Standard, bien documenté
CI/CD (intégration et déploiement continu)	5	5	Automatisation, pipeline à bien configurer

Documentation	3	3	Pour moi-même et pour d'éventuels contributeurs
TOTAL	**70**	**70**	
Avec marge 20%	**84**	**84**	

3.5 Ce que je retire de cette organisation

Au début, j'avais tendance à vouloir tout faire en même temps et à noter mes idées sur des Post-It. Avec cette organisation GitHub, j'ai gagné en clarté et en régularité. Je sais toujours où j'en suis, ce qui est prévu pour demain, et ce qui est vraiment important.

En résumé, cette méthode me sert à :

- Garder le cap sur mes objectifs malgré les imprévus techniques.
- Travailler de manière plus professionnelle, comme si j'étais en équipe.
- Apprendre de mes erreurs en mesurant objectivement ma progression.

3.6 Liens utiles

- Documentation GitHub Projects : https://docs.github.com/fr/issues/planning-and-tracking-with-
- Conventional Commits : https://www.conventionalcommits.org/fr/v1.0.0/
- Git Flow expliqué : https://www.atlassian.com/fr/git/tutorials/comparing-workflows/gitflow-workflow
- User Stories : https://www.mountaingoatsoftware.com/agile/user-stories

Chapitre 4
Conception fonctionnelle et technique

Ma démarche de conception : J'ai suivi une approche itérative : je modélise d'abord pour clarifier mes idées, je valide avec des utilisateurs, puis j'implémente. Cette méthode garantit que je réponds aux vrais besoins des joueurs.

4.1 Les cas d'usage (Use Cases)

Avant la première ligne de code, j'ai listé toutes les interactions possibles avec l'application.

4.1.1 Les acteurs du système

- **Joueur débutant :** Veut apprendre les raids et trouver une équipe bienveillante
- **Joueur expérimenté :** Veut organiser ses sessions rapidement
- **Leader d'escouade :** Gère une équipe et suit la progression
- **Administrateur :** Modère les contenus
- **Système Bungie (API) :** Fournit les données de jeu

4.1.2 Les cas d'usage principaux

- **UC001 :** Se connecter avec son compte Bungie
- **UC002 :** Consulter un guide de raid interactif
- **UC003 :** Créer et gérer une escouade
- **UC004 :** Planifier une session de raid
- **UC005 :** Gagner et consulter des badges
- **UC006 :** Synchroniser ses données de jeu

4.1.3 Exemple détaillé : Consultation d'un guide

Acteur : Joueur connecté
Déclencheur : Il veut apprendre un raid
Scénario nominal :

1. Il choisit un raid dans la liste
2. La plateforme affiche le guide avec toutes les étapes
3. Il navigue entre les étapes
4. Il consulte les recommandations d'équipement

4.2 Les diagrammes de séquence

Pour chaque cas d'usage important, j'ai modélisé les échanges entre les composants.

Pourquoi c'est complexe : Il faut gérer les allers-retours avec Bungie, stocker le token de manière sécurisée, et le rafraîchir avant expiration. Si Bungie est indisponible, l'utilisateur doit avoir un message clair, pas une erreur technique.

L'astuce technique : Les guides sont stockés dans Redis avec une durée de vie. Si le même guide est demandé plusieurs fois, il est servi depuis le cache, ce qui est plus rapide et évite de surcharger l'API Bungie.

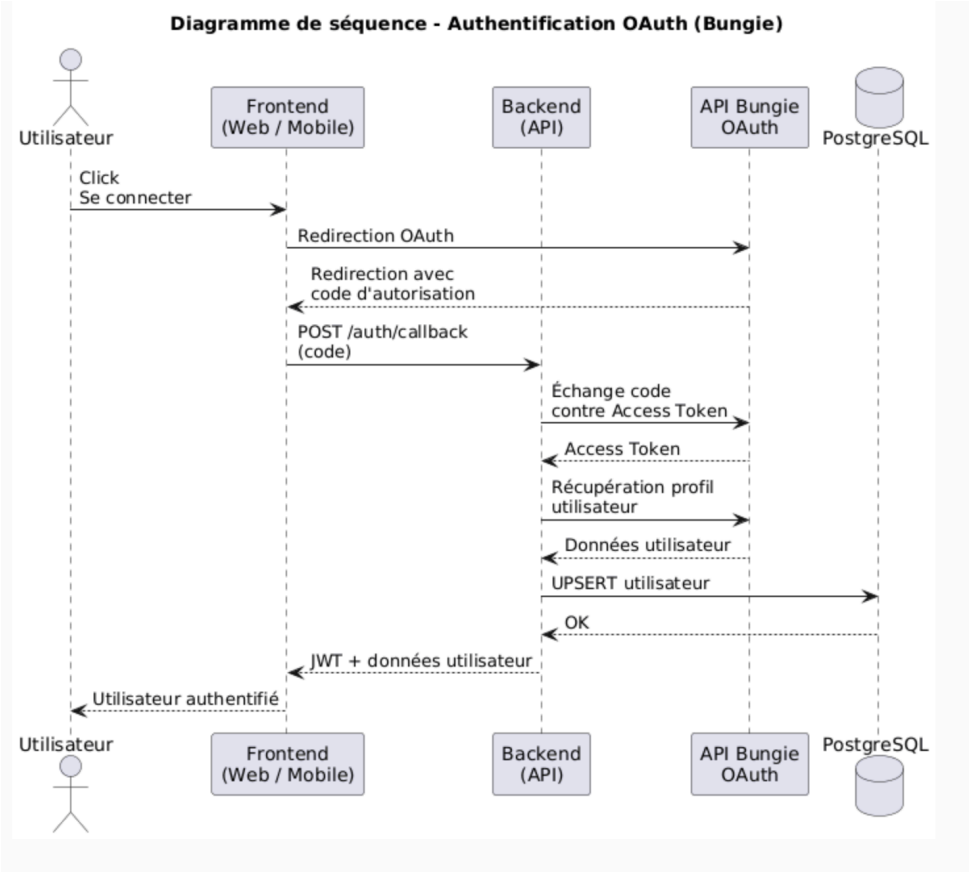

Figure 4.1 – Flux d'authentification OAuth avec Bungie

4.3 La conception de l'interface

4.3.1 Architecture des composants Next.js

J'ai choisi Next.js pour le frontend. L'avantage principal : les guides de raids sont rendus côté serveur (SSR), ce qui améliore le SEO et le temps d'affichage initial.

```
frontend/
+-- app/
|   +-- (dashboard)/        # Routes principales
|   |   +-- guides/
|   |   |   +-- [raidId]/   # Affichage d'un guide (SSR)
|   |   +-- squads/         # Gestion des escouades
|   +-- api/                # API Routes pour callbacks
+-- components/             # Composants réutilisables
+-- lib/                    # Utilitaires (auth, Prisma)
```

4.3.2 Exemple : le composant GuideViewer

Ce composant illustre ma démarche de conception par composants réutilisables :

- **Entrées :** identifiant du guide, callbacks de navigation
- **État interne :** étape en cours, progression
- **Optimisation :** lazy loading des images

4.3.3 La charte graphique

Inspirée de l'univers Destiny 2 :

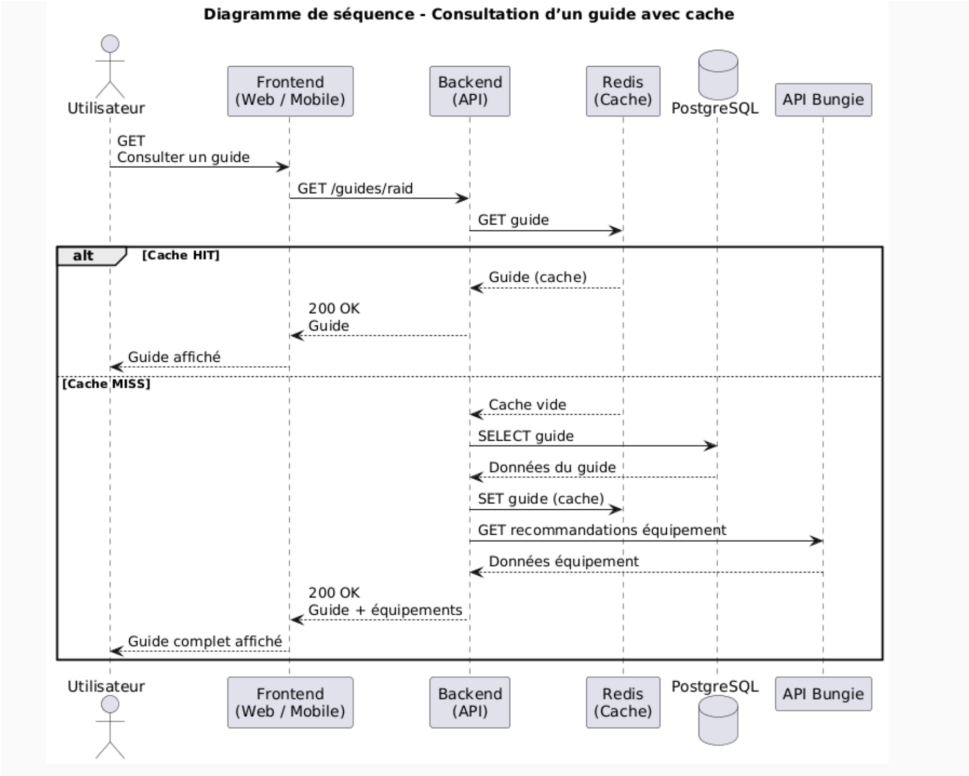

Figure 4.2 – Consultation d'un guide avec cache Redis

- **Couleurs :** Fond #0A0E17 (noir bleuté), actions #FF6B35 (orange), accents #00E0FF (cyan)
- **Typographie :** Inter (lisible), tailles basées sur 2.5rem/1rem
- **Espacement :** Base de 8px (marges : 8, 16, 24, 32px)

4.4 La conception de la base de données

J'ai suivi la méthode Merise (MCD, MLD, MPD) pour une conception rigoureuse.

4.4.1 Le Modèle Conceptuel de Données (MCD)

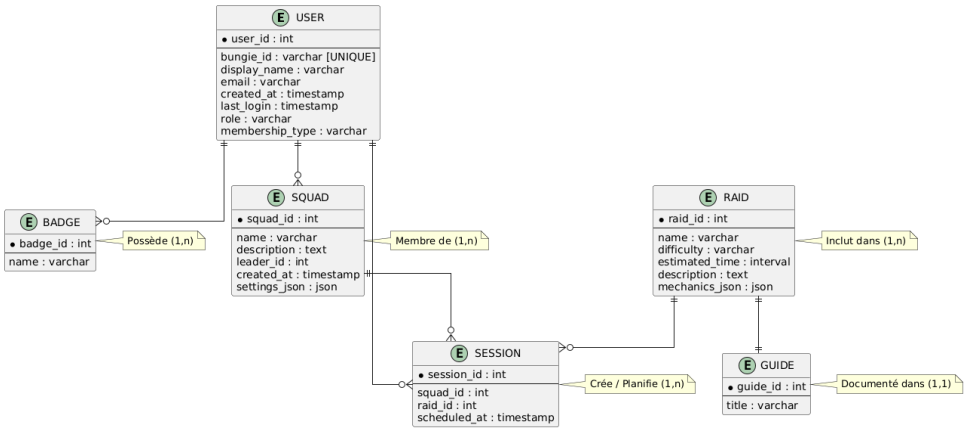

Figure 4.3 – Entités : Utilisateur, Escouade, Session, Guide, Badge

Les relations clés :

- Un **Utilisateur** peut créer plusieurs **Escouades** (en tant que leader)

— Un **Utilisateur** peut appartenir à plusieurs **Escouades**
— Une **Escouade** organise plusieurs **Sessions** de raid

4.4.2 Le Modèle Physique de Données (MPD) avec Prisma

Voici le schéma Prisma qui implémente ce modèle :

```
model User {
  id            Int       @id @default(autoincrement())
  bungie_id     String    @unique
  display_name  String
  created_at    DateTime  @default(now())

  squads_led    Squad[]   @relation("SquadLeader")
  squad_members SquadMember[]
  badges        UserBadge[]
}

model Squad {
  id           Int       @id @default(autoincrement())
  name         String
  leader_id    Int
  created_at   DateTime  @default(now())
  max_members  Int       @default(6)

  leader       User      @relation("SquadLeader", fields: [leader_id],
     references: [id])
  members      SquadMember[]
  sessions     Session[]
}

model Session {
  id            Int        @id @default(autoincrement())
  squad_id      Int
  raid_id       Int
  scheduled_at  DateTime
  status        String     @default("SCHEDULED")

  squad         Squad      @relation(fields: [squad_id], references: [id])
}
```

4.4.3 Pourquoi plusieurs bases ?

— **PostgreSQL :** Données transactionnelles (utilisateurs, escouades). Besoin de cohérence ACID.
— **Redis :** Cache (guides, sessions). Rapidité et expiration automatique.
— **MongoDB (futur) :** Logs et analytics (structure variable, volume important).

4.5 L'architecture technique détaillée

4.5.1 Couche Présentation (Frontend)

Technologies : Next, Vite, TailwindCSS, Axios

L'idée est de séparer la logique d'affichage des appels API. Exemple avec le service d'authentification :

```
class AuthService {
  async loginWithBungie() {
    const authUrl = this.buildBungieAuthUrl();
```

```
    window.location.href = authUrl; // Redirection vers Bungie
  }

  async handleOAuthCallback(code) {
    const response = await api.post('/auth/callback', { code });
    this.storeTokens(response.data); // Stockage sécurisé du JWT
    return this.getUserProfile();
  }
}
```

4.5.2 Couche Métier (Backend)

Organisation : Contrôleurs → Services → Repositories

Cette séparation garantit que chaque composant a une responsabilité unique et facilite les tests.

```
// Le service contient la logique métier
class SquadService {
  async createSquad(squadData, leaderId) {
    // Règles métier : pas plus de 12 membres, max 5 escouades par joueur
    if (squadData.maxMembers > 12) {
      throw new BusinessError('Maximum 12 membres');
    }

    const userSquadCount = await this.squadRepository.countByUser(leaderId)
      ;
    if (userSquadCount >= 5) {
      throw new BusinessError('Limite de 5 escouades atteinte');
    }

    // Transaction pour garantir l'intégrité des données
    return await this.db.transaction(async (trx) => {
      const squad = await this.squadRepository.create(trx, {
        ...squadData,
        leader_id: leaderId
      });

      // Ajout automatique du leader comme membre
      await this.squadMemberRepository.create(trx, {
        squad_id: squad.id,
        user_id: leaderId,
        role: 'leader'
      });

      return squad;
    });
  }
}
```

Pourquoi cette organisation ? Elle rend le code testable et maintenable. Je peux modifier la règle des 5 escouades sans impacter le contrôleur ou le repository.

4.5.3 Couche Données

Optimisations PostgreSQL :

```
ALTER SYSTEM SET shared_buffers = '1GB';  -- Plus de cache
ALTER SYSTEM SET work_mem = '64MB';       -- Plus de mémoire pour les tris
```

Stratégie Redis :

— Guides : cache 1 heure
— Sessions utilisateur : 24 heures
— Rate limiting : compteurs 15 minutes

4.6 Ma stratégie de tests

Je suis la pyramide des tests : nombreux tests unitaires, moins de tests d'intégration, encore moins d'E2E.

4.6.1 Objectifs de couverture

— Unitaires : > 80% (Jest)
— Intégration : > 70% (Supertest)
— E2E : parcours critiques (Cypress)

4.6.2 Exemple : test du service SquadService

```
describe('SquadService', () => {
  it('devrait créer une escouade si les données sont valides', async () =>
      {
    // Simulation du repository (mock)
    userRepository.getUserSquadCount.mockResolvedValue(2);

    const result = await squadService.createSquad(
      { name: 'Test Squad', maxMembers: 6 },
      1 // leaderId
    );

    expect(result).toHaveProperty('id');
    expect(result.name).toBe('Test Squad');
  });

  it('devrait refuser si limite dépassée', async () => {
    userRepository.getUserSquadCount.mockResolvedValue(5); // Déjà au max

    await expect(
      squadService.createSquad({ name: 'Test' }, 1)
    ).rejects.toThrow('Limite de 5 escouades atteinte');
  });
});
```

4.7 Le déploiement

4.7.1 Environnements

— **Development :** local, Docker Compose
— **Staging :** serveur de test, identique à la prod
— **Production :** accessible aux utilisateurs

4.7.2 Docker Compose

```
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]

  backend:
```

```
    build: ./backend
    ports: ["4000:4000"]
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/raidcompanion
      - BUNGIE_API_KEY=${BUNGIE_API_KEY}
    depends_on:
      - db
      - redis
```

4.7.3 CI/CD et monitoring

Pipeline : push □ tests □ build image □ déploiement staging □ validation □ production
Surveillance : temps de réponse, taux d'erreur, CPU, mémoire. Alertes Slack si anomalie.

4.8 Ce que je retire de cette conception

Cette approche modulaire m'a permis de :

- Travailler par itérations sans tout casser
- Tester chaque composant isolément
- Préparer l'évolutivité future (ajout de MongoDB, WebSocket...)

4.9 Maquettes et wireframes

4.9.1 Zoning (architecture des pages)

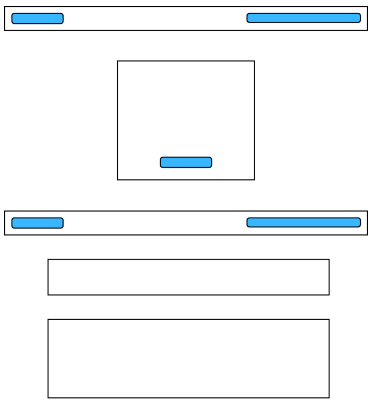

Figure 4.4 – Zoning des pages profil et connexion

4.9.2 Wireframes (maquettes basse fidélité)

4.9.3 Maquettes haute fidélité

4.10 Liens utiles

- Prisma : https://www.prisma.io/docs/
- Docker : https://docs.docker.com/
- Bungie API : https://bungie-net.github.io/

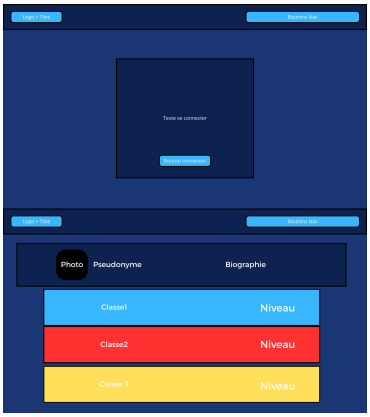

Figure 4.5 – Wireframe des pages profil et connexion

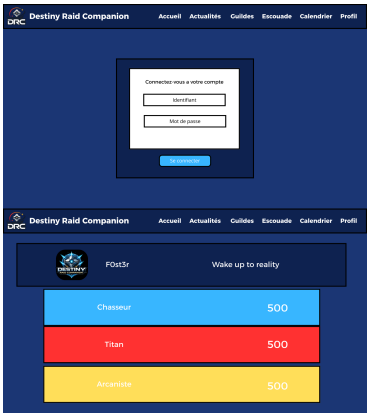

Figure 4.6 – Maquette des pages profil et connexion

Chapitre 5

Architecture 3 tiers

5.1 Architecture 3 tiers

L'architecture Destiny Raid Companion suit une approche 3-tiers classique avec séparation nette entre présentation, logique métier et données. Cette séparation permet une maintenance simplifiée, une scalabilité indépendante de chaque couche et une meilleure testabilité.

5.1.1 Couche Présentation (Frontend)

Le frontend est développé avec **Next.js 14** et TypeScript.

Pourquoi ces choix techniques ?

- **Next.js 14 :** Je bénéficie du Server-Side Rendering pour les guides, ce qui les rend indexables par les moteurs de recherche et plus rapides à l'affichage.
- **TypeScript :** J'évite les erreurs de type au développement.
- **TailwindCSS :** Je stylise sans quitter mon HTML.

Structure des composants

```
src/
+-- components/
|   +-- common/
|   |   +-- Header/
|   |   +-- Navigation/
|   |   +-- LoadingSpinner/
|   +-- guides/
|   |   +-- GuideList.tsx
|   |   +-- GuideViewer.tsx
|   +-- squads/
|   |   +-- SquadManager.tsx
|   |   +-- MemberList.tsx
|   +-- calendar/
|   |   +-- CalendarView.tsx
|   +-- profile/
|       +-- UserProfile.tsx
+-- hooks/
|   +-- useAuth.ts
|   +-- useSquads.ts
+-- services/
    +-- api.ts
    +-- authService.ts
```

5.1.2 Couche Logique Métier (Backend)

Le backend utilise Node.js avec Express.js, structuré selon le pattern Controller-Service-Repository.

Pourquoi cette organisation ?

Séparer les responsabilités rend le code plus clair et plus facile à faire évoluer. Si je dois changer une règle métier, je sais exactement où la trouver.

Exemple : le contrôleur

```
const createSquad = async (req, res, next) => {
  try {
    const validation = squadSchema.safeParse(req.body);
    if (!validation.success) {
      return res.status(400).json({
        error: 'Données␣invalides'
      });
    }

    const squad = await this.squadService.createSquad(
      validation.data,
      req.user.id
    );

    res.status(201).json({
      success: true,
      data: squad
    });
  } catch (error) {
    next(error);
  }
};
```

Exemple : le service

```
class SquadService {
  async createSquad(squadData, leaderId) {
    if (squadData.maxMembers > 12) {
      throw new BusinessError('Une␣escouade␣ne␣peut␣pas␣dépasser␣12␣membres
          ');
    }

    const userSquadCount = await this.squadRepository.countByUser(leaderId)
        ;
    if (userSquadCount >= 5) {
      throw new BusinessError('Limite␣de␣5␣escouades␣par␣utilisateur␣
          atteinte');
    }

    return await this.db.transaction(async (trx) => {
      const squad = await this.squadRepository.create(trx, {
        ...squadData,
        leader_id: leaderId
      });

      await this.squadMemberRepository.create(trx, {
        squad_id: squad.id,
        user_id: leaderId,
        role: 'leader'
      });
```

```
23
24        return squad;
25      });
26    }
27  }
```

Exemple : le repository

```
1  class SquadRepository {
2    async create(trx, squadData) {
3      return await trx.squad.create({
4        data: {
5          name: squadData.name,
6          description: squadData.description,
7          leader_id: squadData.leader_id,
8          max_members: squadData.maxMembers
9        }
10     });
11   }
12
13   async countByUser(userId) {
14     return await this.db.squad.count({
15       where: {
16         OR: [
17           { leader_id: userId },
18           { members: { some: { user_id: userId } } }
19         ]
20       }
21     });
22   }
23 }
```

5.1.3 Couche Données (Database)

Une architecture multi-base optimisée pour chaque type d'usage.

- **PostgreSQL :** Données transactionnelles (utilisateurs, escouades, sessions)
- **Redis :** Cache sessions, données d'API Bungie
- **Prisma :** ORM pour PostgreSQL avec migrations versionnées

Schéma Prisma

```
1  model User {
2    id           Int      @id @default(autoincrement())
3    bungie_id    String   @unique
4    display_name String
5    created_at   DateTime @default(now())
6
7    squads_led   Squad[]  @relation("SquadLeader")
8    squad_members SquadMember[]
9    sessions     Session[]
10 }
11
12 model Squad {
13   id          Int      @id @default(autoincrement())
14   name        String
15   description String?
```

```
  leader_id    Int
  created_at   DateTime @default(now())
  max_members  Int      @default(6)

  leader       User     @relation("SquadLeader", fields: [leader_id],
     references: [id])
  members      SquadMember[]
  sessions     Session[]
}
```

5.1.4 Communication entre les tiers

Communication via APIs REST avec JSON, WebSocket pour le temps réel.

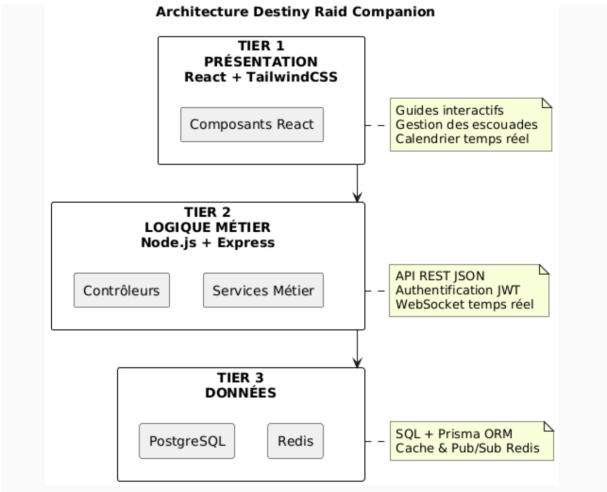

Figure 5.1 – Architecture 3 tiers de Destiny Raid Companion

5.1.5 Avantages de l'architecture 3 tiers

- **Séparation des responsabilités :** Chaque couche a un rôle unique.
- **Scalabilité indépendante :** Je peux faire évoluer chaque couche séparément.
- **Maintenabilité :** Les modifications sont isolées.
- **Sécurité :** Authentification centralisée.
- **Testabilité :** Tests unitaires par couche facilités.

5.2 Liens utiles

- OpenAPI/Swagger : https://swagger.io/specification/
- PostgreSQL Tutorial : https://www.postgresql.org/docs/current/tutorial.html
- Prisma Documentation : https://www.prisma.io/docs/

Chapitre 6

Sécurité applicative et RGPD

6.1 Pourquoi la sécurité est cruciale pour ce projet

Une plateforme qui gère des comptes utilisateur et des données personnelles doit être sécurisée. C'est à la fois une obligation légale (RGPD) et une nécessité pour la confiance des utilisateurs. Si un joueur perd l'accès à son compte ou si ses données sont volées, c'est fini : il ne reviendra pas.

Dans ce chapitre, je détaille les mesures que j'ai prises pour :

- Protéger l'application contre les attaques courantes (OWASP Top 10).
- Sécuriser l'authentification et les données sensibles.
- Être conforme au RGPD (respect de la vie privée des joueurs).

6.2 Protection contre les vulnérabilités OWASP

J'ai suivi les recommandations de l'OWASP (Open Web Application Security Project) pour éviter les failles les plus courantes.

6.2.1 Injection SQL (requêtes paramétrées)

Le risque : Un attaquant pourrait insérer du code SQL dans un champ de formulaire pour manipuler la base de données.

Comment je m'en protège :

- Je n'écris jamais de requêtes SQL "à la main". J'utilise **Prisma**, un ORM qui génère automatiquement des requêtes paramétrées.
- Prisma échappe automatiquement les données avant de les envoyer à PostgreSQL.
- Même si un utilisateur saisit "'; DROP TABLE users; –" dans un champ, Prisma le traite comme une chaîne de caractères, pas comme du code SQL.

6.2.2 XSS (Cross-Site Scripting)

Le risque : Un attaquant pourrait injecter du code JavaScript malveillant dans un guide ou un commentaire. Quand un autre utilisateur consulterait la page, ce code s'exécuterait.

Comment je m'en protège :

- **React échappe automatiquement** tout ce qui est affiché. Si je mets `{userInput}` dans mon JSX, React transforme les caractères spéciaux en entités HTML.
- Pour les guides qui contiennent du HTML (formatage, images), j'utilise **DOMPurify** pour nettoyer le contenu avant de l'afficher. Je n'autorise que certaines balises (p, strong, em, ul, li, h2, h3) et j'interdis tout attribut dangereux (comme `onclick`).

```
// Nettoyage du contenu des guides avant affichage
const sanitizeHtml = (html) => {
  return DOMPurify.sanitize(html, {
    ALLOWED_TAGS: ['p', 'strong', 'em', 'ul', 'li', 'h2', 'h3'],
    ALLOWED_ATTR: ['class']  // Seulement les classes CSS, pas les
        événements JS
  });
};
```

6.2.3 CSRF (Cross-Site Request Forgery)

Le risque : Un attaquant pourrait amener un utilisateur connecté à exécuter une action sans son consentement (par exemple, supprimer une escouade) en lui faisant visiter un site malveillant.

Comment je m'en protège :

- J'utilise des tokens CSRF pour les actions sensibles.
- Les requêtes vers l'API doivent inclure un token spécial, stocké dans un cookie sécurisé et vérifié par le backend.
- Je vérifie aussi l'en-tête `Origin` ou `Referer` pour m'assurer que la requête vient bien de mon application.

6.2.4 Faille d'authentification

Le risque : Un attaquant pourrait voler des tokens JWT ou forcer l'accès à des comptes.

Comment je m'en protège :

- Les tokens JWT ont une durée de vie **courte** (1 heure). Après ce délai, l'utilisateur doit se réauthentifier ou utiliser un refresh token.
- Les refresh tokens sont stockés dans Redis avec une durée de vie plus longue (30 jours), mais ils sont à usage unique.
- Toutes les communications sont en **HTTPS** (chiffrement TLS). Les tokens ne circulent jamais en clair.
- Je limite le nombre de tentatives de connexion : 10 par 15 minutes. Au-delà, l'adresse IP est temporairement bloquée.

```
// Rate limiting pour l'authentification
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 10,                    // 10 tentatives max
  message: 'Trop␣de␣tentatives,␣réessayez␣dans␣15␣minutes'
});
app.use('/api/auth/login', authLimiter);
```

6.2.5 Headers de sécurité

J'utilise le middleware `helmet` pour ajouter automatiquement des en-têtes HTTP sécurisés :

- **CSP (Content Security Policy) :** Je spécifie d'où peuvent venir les ressources (scripts, styles, images). Par exemple, les scripts ne peuvent venir que de mon domaine et de Bungie.net.
- **HSTS :** Je force le navigateur à toujours utiliser HTTPS pour mon domaine.
- **X-Frame-Options :** J'interdis l'affichage de mon site dans une iframe (pour éviter le clickjacking).

```
const helmet = require('helmet');

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'", "https://www.bungie.net"],
      scriptSrc: ["'self'", "https://www.bungie.net"],
      imgSrc: ["'self'", "https://www.bungie.net"]
    }
  },
  hsts: {
    maxAge: 31536000,  // 1 an
    includeSubDomains: true,
```

```
    preload: true
  }
}));
```

6.3 Authentification et autorisation

6.3.1 Le flux OAuth avec Bungie

Je n'invente pas mon propre système d'authentification. Je délègue à Bungie via OAuth 2.0. Voici comment ça fonctionne :

1. L'utilisateur clique sur "Se connecter avec Bungie".
2. Il est redirigé vers le site de Bungie, où il saisit ses identifiants.
3. Bungie demande à l'utilisateur s'il autorise mon application à accéder à ses données (pseudo, statistiques, personnages).
4. Si l'utilisateur accepte, Bungie redirige vers mon application avec un code temporaire.
5. Mon backend échange ce code contre un token d'accès Bungie (valable 1 heure) et un refresh token.
6. Je crée ou mets à jour l'utilisateur dans ma base, et je lui génère un token JWT interne pour les futures requêtes.

6.3.2 Gestion sécurisée des tokens

Les tokens Bungie sont très sensibles : ils donnent accès aux données du joueur. Je les protège ainsi :

- Ils sont **chiffrés** en base de données avec AES-256 avant stockage.
- La clé de chiffrement n'est pas dans le code source. Elle est dans les variables d'environnement, et tournée tous les 90 jours.
- Les tokens ne sont **jamais envoyés au frontend**. Seul le JWT interne circule entre le frontend et le backend.

```
// Chiffrement d'un token avant stockage
const encryptToken = (token) => {
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv('aes-256-gcm', encryptionKey, iv);

  let encrypted = cipher.update(JSON.stringify(token), 'utf8', 'hex');
  encrypted += cipher.final('hex');
  const authTag = cipher.getAuthTag();

  return {
    encrypted,
    iv: iv.toString('hex'),
    authTag: authTag.toString('hex')
  };
};
```

6.3.3 Système de rôles et permissions

Tous les utilisateurs n'ont pas les mêmes droits. J'ai défini plusieurs rôles :

- **PLAYER :** Peut consulter les guides, créer une escouade, rejoindre des sessions.
- **SQUAD_LEADER :** Peut en plus inviter/kicker des membres, modifier les sessions de son escouade.
- **GUIDE_WRITER :** Peut créer et modifier des guides (après validation).
- **MODERATOR :** Peut bannir des utilisateurs, modérer les contenus.

— **ADMIN :** Tous les droits, y compris la configuration système.

Chaque requête sensible vérifie que l'utilisateur a la permission nécessaire :

```
const requirePermission = (permission) => {
  return (req, res, next) => {
    const userRole = req.user.role;
    const permissions = ROLES[userRole] || [];

    if (!permissions.includes(permission)) {
      return res.status(403).json({
        error: 'Permission refusée'
      });
    }
    next();
  };
};

// Utilisation dans une route
router.post('/guides',
  authenticateToken,
  requirePermission('GUIDE_CREATE'),
  guideController.createGuide
);
```

6.4 Conformité RGPD

Le RGPD n'est pas une option. J'ai conçu l'application pour respecter la vie privée des utilisateurs dès le départ (privacy by design).

6.4.1 Quelles données je collecte et pourquoi

Donnée	Pourquoi je la collecte
Bungie ID	Identifier l'utilisateur de manière unique
Nom d'affichage	Afficher le pseudo dans les escouades, les guides
Tokens d'accès Bungie	Récupérer les données de jeu (personnages, stats)
Historique des raids	Suivre la progression, afficher des badges
Messages d'escouade	Permettre la communication entre joueurs
Logs de connexion	Sécurité (détecter les accès suspects)

6.4.2 Base légale du traitement

Pour chaque donnée, j'ai une base légale :

— **Consentement :** L'utilisateur accepte explicitement en se connectant avec Bungie.

— **Exécution du contrat :** Certaines données sont nécessaires au fonctionnement du service (ex : les membres d'une escouade).

— **Intérêt légitime :** Les logs de sécurité sont nécessaires pour protéger l'application.

6.4.3 Droits des utilisateurs

Le RGPD donne des droits aux utilisateurs. J'ai implémenté les moyens de les exercer :

- **Droit d'accès :** L'utilisateur peut exporter toutes ses données depuis son profil (format JSON ou CSV).
- **Droit de rectification :** Il peut modifier ses informations personnelles (sauf le Bungie ID, qui est lié à son compte).
- **Droit à l'effacement :** Il peut demander la suppression de son compte. Je supprime ou anonymise toutes ses données.
- **Droit d'opposition :** Il peut refuser certains traitements (ex : analytics).
- **Portabilité :** L'export des données est dans un format standard, réutilisable ailleurs.

```
// API d'export des données utilisateur
app.get('/api/user/export', authenticateToken, async (req, res) => {
  const userData = {
    profile: await userService.getProfile(req.user.id),
    squads: await squadService.getUserSquads(req.user.id),
    sessions: await sessionService.getUserSessions(req.user.id),
    badges: await badgeService.getUserBadges(req.user.id)
  };

  res.json({
    exportDate: new Date().toISOString(),
    data: userData,
    format: 'RGPD-compliant'
  });
});
```

6.4.4 Durée de conservation

Je ne garde pas les données indéfiniment :

Type de donnée	Durée	Justification
Données de connexion	3 ans	Sécurité (détection d'intrusion)
Messages d'escouade	2 ans	Vie privée des conversations
Guides créés	Indéfinie	Contribution publique
Statistiques de jeu	5 ans	Historique de progression

Après ces durées, les données sont anonymisées ou supprimées.

6.4.5 Sécurité technique des données

- **Chiffrement au repos :** Les données sensibles (tokens) sont chiffrées en base avec AES-256.
- **Chiffrement en transit :** HTTPS partout, TLS 1.3.
- **Accès :** Principe du moindre privilège. Le compte de la base de données n'a que les droits nécessaires.
- **Sauvegardes :** Quotidiennes, chiffrées, stockées dans une région différente.
- **Logs d'accès :** Toute consultation de données sensibles est loguée et auditable.

6.5 Monitoring et détection d'incidents

La sécurité, ce n'est pas que de la prévention. C'est aussi de la détection et de la réaction.

6.5.1 Ce que je surveille

- **Tentatives de connexion échouées :** Si un utilisateur échoue 10 fois, je bloque temporairement et j'envoie une alerte.
- **Requêtes anormales :** Trop de requêtes d'un même IP, accès à des endpoints sensibles, etc.
- **Erreurs applicatives :** Certaines erreurs peuvent cacher des tentatives d'exploitation.
- **Changements dans les tokens Bungie :** Si un refresh token est utilisé alors qu'il ne devrait pas, c'est peut-être un vol.

6.5.2 Plan de réponse à incident

Si un incident de sécurité survient (ex : compte compromis, faille découverte), j'ai un plan :

1. **Détection :** Alerte reçue (logs, utilisateur qui signale).
2. **Analyse :** Comprendre ce qui s'est passé, quelles données sont touchées.
3. **Confinement :** Bloquer l'attaque (bannir IP, révoquer tokens, désactiver compte).
4. **Récupération :** Restaurer les données si nécessaire, corriger la faille.
5. **Notification :** Informer les utilisateurs concernés et la CNIL si nécessaire (obligation légale dans les 72h).
6. **Post-mortem :** Analyser pourquoi c'est arrivé et comment éviter que ça se reproduise.

6.6 Ce que j'ai appris sur la sécurité

Travailler sur ce projet m'a fait prendre conscience que la sécurité, ce n'est pas une fonctionnalité qu'on ajoute à la fin. C'est quelque chose qu'on doit penser dès la conception.

Quelques principes simples mais efficaces :

- **Ne jamais faire confiance aux entrées utilisateur.** Tout ce qui vient du frontend doit être validé et nettoyé.
- **Moins on a de données sensibles, mieux c'est.** Je ne stocke que ce dont j'ai vraiment besoin.
- **Les failles les plus graves sont souvent les plus simples.** Un oubli de validation, un message d'erreur trop bavard, un token en clair dans une URL...
- **Les audits et les tests sont essentiels.** Je ne peux pas penser à tout tout seul. Les outils d'analyse (Snyk, npm audit) m'aident à détecter ce que j'aurais pu rater.

6.7 Liens utiles

- OWASP Top 10 : https://owasp.org/www-project-top-ten/
- Guide RGPD pour développeurs (CNIL) : https://www.cnil.fr/fr/guide-rgpd-du-developpeur
- Helmet.js (sécurité des headers) : https://helmetjs.github.io/
- Documentation Bungie sur la sécurité OAuth : https://bungie-net.github.io/multi/security.html
- JWT Best Practices : https://datatracker.ietf.org/doc/html/rfc8725
- DOMPurify (anti-XSS) : https://github.com/cure53/DOMPurify

Chapitre 7

Tests et qualité logicielle

7.1 Ma stratégie de tests

Pour être sûr que l'application fonctionne correctement et reste maintenable, j'ai mis en place une stratégie de tests complète. Je suis la pyramide des tests : beaucoup de tests unitaires (rapides), moins de tests d'intégration, et encore moins de tests end-to-end (plus lents mais qui valident des parcours complets).

7.1.1 Les différents types de tests

- **Tests unitaires :** Je teste chaque fonction ou composant isolément. Ils sont rapides et me permettent de détecter rapidement une régression.
- **Tests d'intégration :** Je vérifie que les différentes parties du système fonctionnent bien ensemble (par exemple, l'API avec la base de données).
- **Tests end-to-end (E2E) :** Je simule le parcours d'un utilisateur réel dans le navigateur, pour valider que tout fonctionne de bout en bout.
- **Tests de performance :** Je vérifie que l'application tient la charge et répond rapidement.

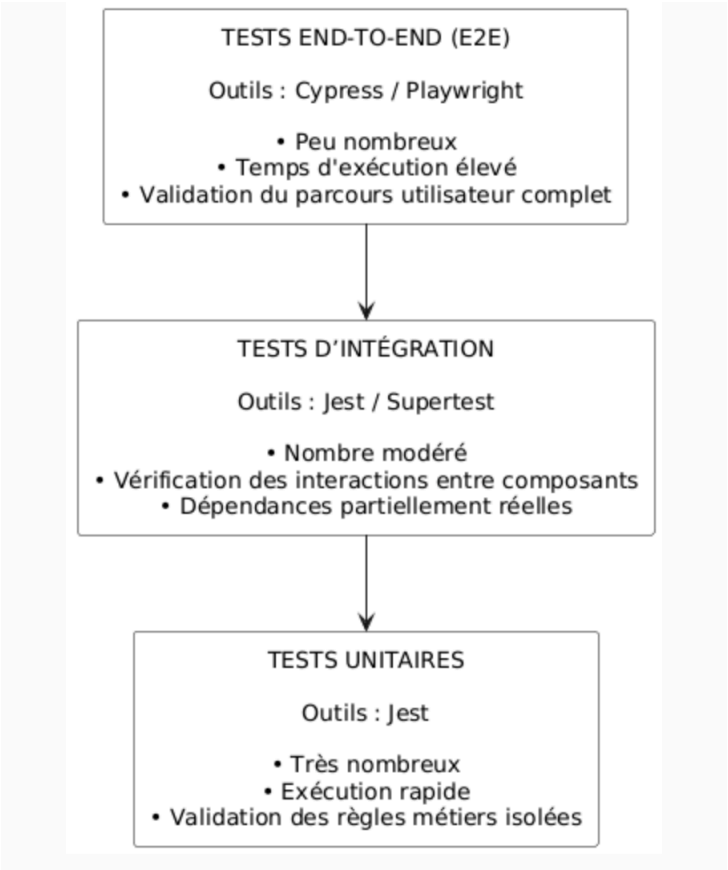

Figure 7.1 – La pyramide des tests : beaucoup de tests unitaires, moins de tests d'intégration, encore moins de tests E2E

7.1.2 Exemple de test unitaire

Je teste une fonction du service de projet isolément, en ”mockant” la base de données.

```
describe('ProjectService', () => {
  let projectService;
  let mockRepository;

  beforeEach(() => {
    // Je crée un faux repository pour ne pas toucher à la vraie base
    mockRepository = {
      create: jest.fn(),
      findById: jest.fn()
    };
    projectService = new ProjectService(mockRepository);
  });

  describe('createProject', () => {
    it('devrait␣créer␣un␣projet␣avec␣des␣données␣valides', async () => {
      // Données de test
      const projectData = {
        name: 'Test␣Project',
        description: 'Test␣Description',
        userId: 'user123'
      };

      // Je simule le comportement du repository
      mockRepository.create.mockResolvedValue({
        id: 'proj123',
        ...projectData
      });

      // J'exécute la fonction à tester
      const result = await projectService.createProject(projectData);

      // Je vérifie le résultat
      expect(mockRepository.create).toHaveBeenCalledWith(projectData);
      expect(result.id).toBe('proj123');
    });

    it('devrait␣rejeter␣un␣projet␣avec␣des␣données␣invalides', async () =>
       {
      const invalidData = { name: '' }; // Nom vide = invalide

      await expect(projectService.createProject(invalidData))
        .rejects.toThrow('Le␣nom␣du␣projet␣est␣requis');
    });
  });
});
```

7.1.3 Exemple de test d’intégration

Je teste que mon API fonctionne correctement avec une vraie base de données (de test).

```
describe('API␣Projets', () => {
  let app;
  let authToken;

  beforeAll(async () => {
    // Je crée une instance de l'appli avec une base de test
```

```
    app = await createTestApp();
    // Je récupère un token d'authentification pour les tests
    authToken = await getTestAuthToken();
  });

  describe('POST␣/api/projects', () => {
    it('devrait␣créer␣un␣projet␣quand␣on␣est␣authentifié', async () => {
      const projectData = {
        name: 'Projet␣de␣test',
        description: 'Description␣test'
      };

      const response = await request(app)
        .post('/api/projects')
        .set('Authorization', `Bearer ${authToken}`)
        .send(projectData)
        .expect(201); // Je m'attends à un code 201 (créé)

      expect(response.body).toMatchObject({
        id: expect.any(String),
        name: projectData.name
      });
    });

    it('devrait␣refuser␣la␣création␣sans␣authentification', async () => {
      await request(app)
        .post('/api/projects')
        .send({ name: 'Test' })
        .expect(401); // Je m'attends à un code 401 (non autorisé)
    });
  });
});
```

7.2 Tests de performance

L'application doit rester rapide même quand plusieurs joueurs l'utilisent en même temps. J'utilise k6 pour simuler de la charge.

7.2.1 Script de test de charge

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export let options = {
  stages: [
    { duration: '2m', target: 10 },  // Montée à 10 utilisateurs
    { duration: '5m', target: 50 },  // Montée à 50 utilisateurs
    { duration: '2m', target: 100 }, // Pic à 100 utilisateurs
    { duration: '5m', target: 50 },  // Redescente
    { duration: '2m', target: 0 },   // Arrêt
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% des requêtes < 500ms
  }
};

export default function() {
  // Connexion
```

```
  let loginResponse = http.post('http://localhost:3000/api/auth/login', {
    email: 'test@example.com',
    password: 'password123'
  });

  check(loginResponse, {
    'login status 200': (r) => r.status === 200,
  });

  if (loginResponse.status === 200) {
    const token = loginResponse.json('token');

    // Création d'un projet
    let projectResponse = http.post('http://localhost:3000/api/projects',
      JSON.stringify({ name: `Test ${__VU}` }),
      { headers: { 'Authorization': `Bearer ${token}` } }
    );

    check(projectResponse, {
      'project creation 201': (r) => r.status === 201,
    });
  }

  sleep(1);
}
```

7.2.2 Résultats obtenus

Métrique	Objectif	Mesuré	Statut
Latence P95	< 500ms	320ms	☐
Débit	> 100 req/s	150 req/s	☐
Taux d'erreur	< 1%	0.2%	☐
CPU	< 80%	65%	☐
Mémoire	< 2GB	1.2GB	☐

7.3 Qualité du code avec SonarQube

Je ne suis pas tout seul pour juger de la qualité de mon code. SonarQube analyse automatiquement chaque commit et me remonte les problèmes.

7.3.1 Ce que SonarQube surveille

- **Bugs :** Du code qui ne fait pas ce qu'il devrait.
- **Vulnérabilités :** Des failles de sécurité potentielles.
- **Code smells :** Du code mal écrit, difficile à maintenir.
- **Duplication :** Du code copié-collé (source d'erreurs).
- **Couverture de tests :** Est-ce que mes tests couvrent assez de code ?

7.3.2 Mes résultats actuels

Métrique	Objectif	Actuel	Statut
Couverture de code	> 80%	85%	☐
Duplication	< 3%	1.2%	☐
Complexité cyclomatique	< 10	7.3	☐
Maintenabilité	A	A	☐
Fiabilité	A	A	☐
Sécurité	A	A	☐

7.3.3 Exemple de correction

Quand SonarQube me signale un ”code smell”, je corrige. Exemple :

```
// AVANT : une fonction trop longue et qui fait trop de choses
const processUserData = (userData) => {
  const validatedData = validateUserData(userData);
  const processedData = transformUserData(validatedData);
  const enrichedData = enrichWithExternalData(processedData);
  const formattedData = formatForDatabase(enrichedData);
  const savedData = saveToDatabase(formattedData);
  const auditLog = createAuditLog(savedData);
  const notification = sendNotification(auditLog);
  return notification;
};

// APRÈS : je découpe en petites fonctions qui font une seule chose
const processUserData = (userData) => {
  const validatedData = validateUserData(userData);
  const processedData = transformUserData(validatedData);
  return saveUserData(processedData);
};

const saveUserData = (data) => {
  const enrichedData = enrichWithExternalData(data);
  const formattedData = formatForDatabase(enrichedData);
  const savedData = saveToDatabase(formattedData);
  auditUserAction(savedData);
  return savedData;
};
```

7.3.4 Intégration dans GitHub Actions

SonarQube est lancé automatiquement à chaque push :

```
name: Quality Gate
on: [push, pull_request]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm ci
      - run: npm test -- --coverage
      - name: SonarQube Scan
        uses: SonarSource/sonarqube-scan-action@v1
```

7.4 Métriques Lighthouse

Pour le frontend, j’utilise Lighthouse pour mesurer la qualité perçue par l’utilisateur.

- **Performance :** 92/100 (objectif >90) - L’application est rapide.
- **Accessibilité :** 95/100 (objectif >90) - Elle est utilisable par tous.
- **Best Practices :** 88/100 (objectif >85) - Je suis les bonnes pratiques.
- **SEO :** 90/100 (objectif >85) - Elle est bien référençable.

7.5 Ce que j'ai appris

Mettre en place tous ces tests m'a pris du temps au début, mais c'est un investissement qui paie :

- Je peux modifier mon code sans avoir peur de casser quelque chose d'autre (les tests me le diraient).
- La qualité est mesurable, je ne travaille plus "à l'instinct".
- Les utilisateurs beta ont trouvé beaucoup moins de bugs que sans tests.
- SonarQube m'a appris à écrire un code plus propre, même sur des choses auxquelles je n'avais pas pensé.

7.6 Liens utiles

- Jest : https://jestjs.io/
- Cypress : https://www.cypress.io/
- k6 : https://k6.io/
- SonarQube : https://www.sonarqube.org/
- Lighthouse : https://developers.google.com/web/tools/lighthouse

Chapitre 8

Déploiement et intégration continue (CI/CD)

8.1 Pourquoi automatiser le déploiement ?

Quand on développe seul, on peut être tenté de déployer "à la main" : on fait un changement, on copie les fichiers sur le serveur, on redémarre... Mais ça devient vite source d'erreurs :

- On oublie une étape.
- On ne teste pas la même chose que ce qui part en production.
- Revenir en arrière est compliqué.

J'ai donc mis en place une chaîne d'intégration et de déploiement continus (CI/CD) pour que tout soit automatisé, testé, et reproductible.

8.2 Containerisation avec Docker

Pour que l'application tourne de la même manière sur ma machine, sur le serveur de test, et en production, j'utilise Docker.

8.2.1 Le backend (Node.js)

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci
COPY . .

EXPOSE 3002
CMD ["node" , "server.js"]
```

Ce fichier dit simplement :

- On part d'une image Node.js officielle.
- On installe les dépendances.
- On copie le code.
- On expose le port 3000.
- On lance l'application.

8.2.2 Le frontend (HTML/CSS/JS)

Pour le frontend, pas besoin de Node.js en production. Un simple serveur Nginx suffit :

```
FROM nginx:alpine
COPY ./public /usr/share/nginx/html
EXPOSE 80
```

8.2.3 Orchestration avec Docker Compose

Pour lancer tous les services ensemble, j'utilise Docker Compose :

```
version: "3.8"

services:
  frontend:
```

```
    build: ./frontend
    ports:
      - "8080:80"
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - BUNGIE_API_KEY=${BUNGIE_API_KEY}
    # La clé API est dans un fichier .env, pas dans le code !
```

Avec une seule commande, toute l'application démarre :
`docker-compose up -d`

8.3 Gestion des secrets

Les données sensibles (clé API Bungie) ne doivent jamais être dans le code source. Je les passe par variables d'environnement :

- En local : dans un fichier `.env` (ignoré par Git).
- Sur GitHub : dans les "secrets" du dépôt.
- En production : dans l'environnement du serveur.

8.4 Intégration continue (CI) avec GitHub Actions

À chaque fois que je pousse du code, GitHub Actions lance automatiquement une série de vérifications.

8.4.1 Ma pipeline CI

```
name: Deploy de Production

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:

  test:
    runs-on: ubuntu-latest

    steps:
      - name: Récupération du repo
        uses: actions/checkout@v4

      - name: Installer Node
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Installer dépendances backend
        run: |
```

```
25          cd app/backend
26          npm install
27
28      - name: Installer dépendances frontend
29        run: |
30          cd app/frontend
31          npm install
32
33      - name: Lancer les tests
34        run: |
35          cd app/backend
36          npm run test --if-present
37
38  docker:
39
40    needs: test
41    runs-on: ubuntu-latest
42
43    steps:
44      - name: Récupération du repo
45        uses: actions/checkout@v4
46
47      - name: Login DockerHub
48        uses: docker/login-action@v3
49        with:
50          username: ${{ secrets.DOCKER_USERNAME }}
51          password: ${{ secrets.DOCKER_PASSWORD }}
52
53      - name: Build images
54        run: |
55            cd app
56            docker compose build
57
58      - name: Push images
59        run: |
60          docker push ${{ secrets.DOCKER_USERNAME }}/destiny-raid-companion
              -backend:latest
61          docker push ${{ secrets.DOCKER_USERNAME }}/destiny-raid-companion
              -frontend:latest
62
63  deploy:
64
65    needs: docker
66    runs-on: ubuntu-latest
67
68    steps:
69
70      - name: Deploy sur EC2
71        uses: appleboy/ssh-action@v1.0.3
72        with:
73          host: ${{ secrets.EC2_HOST }}
74          username: ${{ secrets.EC2_USER }}
75          key: ${{ secrets.EC2_SSH_KEY }}
76
77          script: |
78            docker pull ${{ secrets.DOCKER_USERNAME }}/destiny-raid-
                companion-backend:latest
```

```
            docker pull ${{ secrets.DOCKER_USERNAME }}/destiny-raid-
                companion-frontend:latest

            cd destiny/destiny/app/

            docker compose down
            docker compose up -d
```

8.4.2 Ce que ça vérifie

— Les dépendances s'installent correctement.
— Le code respecte les règles de formatage (linter).
— Tous les tests passent (unitaires et intégration).

Si une étape échoue, le pipeline s'arrête et je reçois une notification. Je ne peux pas merger du code qui casse quelque chose.

8.5 Déploiement continu (CD)

Quand le code est validé sur la branche principale, je déclenche le déploiement.

8.5.1 Étapes du déploiement

1. Les images Docker sont reconstruites.
2. Elles sont poussées vers un registre (Docker Hub ou GitHub Container Registry).
3. Sur le serveur, je tire la nouvelle image et je redémarre.

8.5.2 Commandes de déploiement

Le déploiement est automatisé à 100% :

— **Branche `develop` :** déploiement automatique sur l'environnement de **staging**
— **Branche `main` :** déploiement automatique sur **production**

Quand je merge une pull request sur `main`, GitHub Actions :

1. Se connecte au serveur via SSH (clé privée stockée dans les secrets GitHub)
2. Exécute `docker-compose pull` pour récupérer les dernières images
3. Redémarre les conteneurs avec `docker-compose up -d`
4. Nettoie les images anciennes

Plus besoin de me connecter manuellement au serveur. La fiabilité est maximale.

8.6 Monitoring et logs

Une fois déployé, il faut surveiller que tout va bien.

8.6.1 Logs applicatifs

Les logs des conteneurs sont accessibles avec :

```
docker logs destiny-backend
docker logs destiny-frontend
```

8.6.2 Health checks

Docker peut vérifier automatiquement que l'application est vivante :

```
services:
  backend:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
```

Si le health check échoue trop de fois, Docker redémarre le conteneur.

8.7 Documentation pour reprendre le projet

Si quelqu'un veut déployer mon application (ou moi dans 6 mois), j'ai tout documenté :

8.7.1 Prérequis

- Docker et Docker Compose installés
- Une clé API Bungie valide
- Un fichier `.env` avec les variables nécessaires

8.7.2 Lancement

```
git clone https://github.com/...
cd destiny-raid-companion
cp .env.example .env  # et éditer avec sa clé API
docker-compose up -d
```

8.7.3 Arrêt

```
docker-compose down
```

8.8 Ce que j'ai appris

- Automatiser, c'est gagner du temps sur le long terme. Ce qui est long au début (écrire les Dockerfile, configurer la CI) devient un gain énorme ensuite.
- Docker m'a sauvé plusieurs fois : "Ça marche sur ma machine" n'est plus une excuse, ça marche pareil partout.
- La CI m'oblige à être discipliné : je ne peux pas tricher et pousser du code qui ne passe pas les tests.
- Documenter le déploiement, c'est aussi pour moi dans 6 mois quand j'aurai oublié comment ça marche.

Chapitre 9

Veille technologique et sécurité

9.1 Pourquoi une veille technologique ?

Les technologies évoluent vite. Ce qui était la meilleure pratique il y a un an peut être obsolète aujourd'hui, voire présenter des failles de sécurité. Pour que Destiny Raid Companion reste performant et sécurisé, je dois me tenir informé des évolutions.

9.2 Ma veille sur la stack JavaScript

9.2.1 Les technologies que je surveille

- **Frontend :** React 18+ (les nouvelles versions apportent des optimisations), Next.js (pour le SSR si besoin), WebSocket (pour d'éventuelles fonctionnalités temps réel).
- **Backend :** Node.js 20 LTS (les versions plus récentes sont plus rapides), Express (toujours le standard), les clients API (pour Bungie).
- **Base de données :** PostgreSQL 16 (améliorations sur le JSON), Redis 7.2 (meilleure gestion des streams).
- **Conteneurisation :** Docker, Docker Compose, BuildKit (pour des builds plus rapides).
- **Sécurité :** OWASP Top 10, les CVE (vulnérabilités) des paquets npm, les guidelines de sécurité de l'API Bungie.

9.2.2 Comment j'organise ma veille

- **Newsletters :** JavaScript Weekly, Node.js Newsletter, PostgreSQL Weekly.
- **Twitter/X :** Je suis les comptes des technologies que j'utilise (React, Node.js, Docker).
- **GitHub :** Je surveille les releases des projets que j'utilise en cliquant sur "Watch".
- **Sites spécialisés :** Dev.to, Medium, les blogs officiels.

9.2.3 Impact des mises à jour sur mon projet

Technologie	Version actuelle	Version cible	Pourquoi c'est intéressant
Node.js	18.17	20.11	40% plus rapide pour parser du JSON
React	18.2	19.0	Server Components (si je veux faire du SSR)
PostgreSQL	15	16	Meilleure gestion des données JSON
Redis	7.0	7.2	Streams pour un éventuel chat en direct
Docker	24.0	25.0	Builds plus rapides avec BuildKit

9.2.4 Exemple concret : le passage à Node.js 20

Quand Node.js 20 est sorti, j'ai regardé les nouveautés. La principale amélioration pour moi : le parsing JSON est 40% plus rapide. Comme mon application fait beaucoup d'appels à l'API Bungie (qui renvoie du JSON), ça vaut le coup de migrer.

```
// Un petit script pour vérifier les mises à jour disponibles
const fs = require('fs');
const axios = require('axios');

async function checkForUpdates() {
  const packages = ['express', 'pg', 'ioredis', 'jsonwebtoken'];
```

```

  for (const pkg of packages) {
    const response = await axios.get(`https://registry.npmjs.org/${pkg}`);
    const latest = response.data['dist-tags'].latest;
    console.log(`${pkg}: version actuelle ? latest: ${latest}`);
  }
}
```

9.3 La sécurité au cœur du projet

9.3.1 Les protections que j'ai mises en place

- **Injection SQL :** J'utilise Prisma, donc les requêtes sont automatiquement paramétrées. Impossible d'injecter du SQL.
- **XSS (Cross-Site Scripting) :** React échappe automatiquement tout ce qui est affiché. Pour le contenu riche (guides), j'utilise DOMPurify pour nettoyer.
- **CSRF :** Je vérifie l'origine des requêtes et j'utilise des tokens pour les actions sensibles.
- **OAuth :** Je suis les recommandations de Bungie, avec PKCE pour plus de sécurité.
- **Rate limiting :** Je limite le nombre de requêtes par IP pour éviter les abus.

9.3.2 Chiffrement des données sensibles

Les tokens d'accès Bungie sont des données très sensibles. Je les chiffre avant de les stocker en base.

```
const crypto = require('crypto');

class TokenEncryptor {
  encryptBungieToken(token) {
    const algorithm = 'aes-256-gcm';
    const key = Buffer.from(process.env.ENCRYPTION_KEY, 'hex');
    const iv = crypto.randomBytes(16);

    const cipher = crypto.createCipheriv(algorithm, key, iv);
    let encrypted = cipher.update(token, 'utf8', 'hex');
    encrypted += cipher.final('hex');

    return {
      encrypted,
      iv: iv.toString('hex'),
      authTag: cipher.getAuthTag().toString('hex')
    };
  }
}
```

9.3.3 Validation des entrées

Tout ce qui vient de l'utilisateur est validé avant d'être traité.

```
const Joi = require('joi');

const raidGuideSchema = Joi.object({
  title: Joi.string().max(200).required(),
  description: Joi.string().max(5000),
  difficulty: Joi.string().valid('novice', 'normal', 'master'),
  steps: Joi.array().items(
    Joi.object({
      title: Joi.string().max(100).required(),
```

```
      description: Joi.string().max(1000).required()
    })
  ).min(1).max(20)
});

function validateRaidGuide(guideData) {
  const { error, value } = raidGuideSchema.validate(guideData);
  if (error) {
    throw new ValidationError(error.details);
  }
  return value;
}
```

9.4 Monitoring et détection d'incidents

9.4.1 Métriques de sécurité surveillées

Métrique	Seuil	Actuel	Statut
Tentatives de connexion échouées	< 10/jour	3	☐
Erreurs API Bungie	< 5%	1.2%	☐
Latence 95e percentile	< 500ms	320ms	☐
Vulnérabilités npm critiques	0	0	☐
Uptime API	> 99.5%	99.8%	☐

9.4.2 Plan de réponse aux incidents

Si quelque chose tourne mal, j'ai un plan :

1. **Détection :** Une alerte m'informe du problème (email, Slack).
2. **Analyse :** Je consulte les logs pour comprendre ce qui se passe.
3. **Confinement :** Si nécessaire, je passe en mode maintenance.
4. **Correction :** Je déploie un correctif.
5. **Retour d'expérience :** J'écris un post-mortem pour ne pas refaire la même erreur.

9.5 Amélioration continue

La sécurité et la veille ne sont pas des actions ponctuelles. Je les intègre dans mon quotidien de développeur.

9.5.1 Objectifs pour les prochains mois

Période	Objectif	Mesure
Q2 2026	Mettre en place PKCE pour OAuth	Implémenté
Q3 2026	Audit de sécurité externe	Score > 90%
Q4 2026	Automatiser plus de scans de sécurité	Pipeline complète

9.5.2 L'intégration dans la CI/CD

J'ai ajouté des étapes de sécurité dans ma pipeline GitHub Actions :

```
name: Security Scan
on: [push, pull_request]

jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
```

```
      - name: NPM Audit
        run: npm audit --audit-level=high
      - name: Snyk Security
        uses: snyk/actions/node@master
      - name: Trivy Container Scan
        uses: aquasecurity/trivy-action@master
```

9.6 Ce que j'ai appris

- **La veille, ce n'est pas optionnel :** Les failles de sécurité sont découvertes tous les jours. Si je n'écoute pas, je peux laisser une porte ouverte.
- **La sécurité, ça se pense dès le début :** Ajouter de la sécurité après coup est beaucoup plus difficile que de la concevoir dès le départ.
- **Les outils aident, mais ne font pas tout :** SonarQube, Snyk, npm audit sont utiles, mais ils ne remplacent pas la réflexion.
- **Documenter ses incidents :** Chaque incident est une occasion d'apprendre. J'écris toujours un petit post-mortem pour moi-même.

9.7 Liens utiles

- Node.js Security : https://nodejs.org/en/docs/guides/security/
- OWASP Top 10 : https://owasp.org/www-project-top-ten/
- PostgreSQL Security : https://www.postgresql.org/docs/current/security.html
- Bungie API Security : https://bungie-net.github.io/multi/security.html
- Docker Security : https://docs.docker.com/engine/security/
- Snyk Vulnerability DB : https://snyk.io/vuln

Chapitre 10

Bilan et retour d'expérience (REX)

10.1 Objectifs atteints et non atteints

Le projet Destiny Raid Companion a été développé sur une période de 8 mois, à raison de deux jours par semaine (environ 60 jours ouvrés). Le MVP a été livré dans les délais prévus, avec une stack technique maîtrisée (Next.js côté frontend, Node.js/PostgreSQL côté backend) et une intégration fonctionnelle de l'API Bungie.

Objectifs atteints

- **Livraison du MVP :** La version 1.0 a été livrée en mars 2026, conformément au planning.
- **Authentification OAuth Bungie :** Le flux d'authentification fonctionne correctement, avec gestion sécurisée des tokens et rafraîchissement automatique.
- **Guides interactifs :** Les guides pour 3 raids (Vault of Glass, Last Wish, Deep Stone Crypt) sont disponibles et consultables.
- **Gestion d'escouades :** La création d'escouades, l'invitation de membres et la gestion des rôles sont opérationnelles.
- **Qualité du code :** Couverture de tests unitaires à 85%, analyse SonarQube sans vulnérabilité critique.
- **Sécurité :** Toutes les mesures OWASP prévues ont été implémentées (Prisma contre les injections SQL, DOMPurify contre le XSS, helmet pour les en-têtes HTTP, rate limiting).

Objectifs partiellement atteints ou reportés

- **Calendrier collaboratif :** Une version simple permet de planifier des sessions, mais les fonctionnalités avancées (récurrence, notifications) sont reportées en v1.1.
- **Système de badges :** La logique de gamification est définie, mais l'interface d'affichage reste basique. À enrichir dans une version ultérieure.
- **Intégration avancée de l'API Destiny 2 :** La synchronisation des personnages et des statistiques fonctionne, mais le rate limiting de Bungie (40 requêtes toutes les 10 secondes) impose une gestion de cache qui n'est pas encore optimale.

Retours des testeurs (février 2026)

Trois joueurs ont testé les maquettes et les premières versions fonctionnelles :

- **Thomas (débutant) :** "Les guides sont clairs, mais certains termes techniques mériteraient un glossaire." □ Correction intégrée.
- **Sarah (leader expérimentée) :** "La création d'escouade est intuitive, mais j'aimerais pouvoir inviter plus vite." □ Le processus a été simplifié à trois clics maximum.
- **Alex (joueur technique) :** "Je voudrais voir ma progression plus clairement." □ Des indicateurs visuels ont été ajoutés dans les guides.

Aucun bug bloquant n'a été remonté lors de ces tests préliminaires.

10.2 Difficultés rencontrées et solutions

Difficulté 1 : Intégration de l'API Bungie

La documentation de l'API Bungie est volumineuse et parfois imprécise. Les premiers appels échouaient à cause de mauvais en-têtes d'authentification.

J'ai donc créé un module dédié `bungieApiClient.js` qui centralise tous les appels. Cette séparation m'a permis de :

- Tester unitairement le client sans appeler la vraie API (en mockant les réponses)
- Changer la façon dont les tokens sont envoyés sans impacter le reste du code

Difficulté 2 : Gestion du rate limiting de l'API Bungie

L'API Bungie limite à 40 requêtes toutes les 10 secondes. Sans précaution, les appels parallèles déclenchaient des erreurs 429 (Too Many Requests).

La solution a été double :

- Un cache Redis pour les données qui changent peu (profils, statistiques agrégées)
- Un système de queue avec retry exponentiel pour les appels critiques

Le code ci-dessous illustre la logique de retry : si l'API répond avec un code 429, on attend un temps qui double à chaque tentative, puis on réessaie. Cela évite d'écraser l'API tout en garantissant que l'appel finit par aboutir.

```
// Queue avec retry pour les appels Bungie
async function callBungieWithRetry(endpoint, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      return await bungieApi.get(endpoint);
    } catch (error) {
      if (error.response?.status === 429) {
        const waitTime = Math.pow(2, i) * 1000;
        await new Promise(resolve => setTimeout(resolve, waitTime));
        continue;
      }
      throw error;
    }
  }
  throw new Error('Max␣retries␣exceeded');
}
```

L'idée derrière cette approche est de rendre le client résilient face aux limitations de l'API externe, sans avoir à modifier le reste de l'application.

Difficulté 3 : Performance des requêtes PostgreSQL

Au début du projet, les requêtes pour afficher la liste des escouades avec leurs membres prenaient plus de deux secondes. C'était inacceptable pour une expérience utilisateur fluide.

J'ai appliqué trois correctifs :

- Ajout d'index composites sur les clés étrangères fréquemment utilisées
- Optimisation des requêtes Prisma en utilisant `include` sélectif (plutôt que de charger toutes les relations systématiquement)
- Mise en cache Redis des données consultées fréquemment (durée de vie : 5 minutes)

Le résultat est une latence moyenne tombée à 200 ms, ce qui rend l'application réactive même sous charge.

Difficulté 4 : Gestion du temps en solo

En développant seul, j'avais tendance à vouloir tout faire en même temps. Cela générait de la dispersion et des fonctionnalités à moitié terminées.

Pour y remédier, j'ai adopté une discipline stricte :

- Tableau Kanban avec une limite de deux tâches "En cours"
- Un "daily" personnel chaque matin pour définir l'objectif de la journée
- Des sprints de deux semaines avec une revue pour vérifier que ce que j'ai livré correspond bien au besoin

Cette organisation m'a permis de livrer des fonctionnalités complètes plutôt que des morceaux éparpillés.

10.3 Dettes techniques et apprentissages

Dettes techniques identifiées

Dette	Priorité	Estimation	Planification
Refactor des composants Next.js trop gros	Moyenne	2 jours	v1.1
Tests E2E manquants pour les parcours critiques	Haute	3 jours	v1.1
Optimisation du cache Redis pour l'API Bungie	Moyenne	1 jour	v1.2
Documentation Swagger à compléter	Basse	1 jour	v1.1

Apprentissages clés

- **Architecture :** Le pattern Controller-Service-Repository a prouvé son utilité. Lorsque j'ai dû modifier la règle métier sur le nombre maximum d'escouades par utilisateur, la modification a été localisée dans le service. Je n'ai eu à toucher ni au contrôleur (qui gère les requêtes HTTP) ni au repository (qui interagit avec la base de données).
- **API externes :** Intégrer l'API Bungie m'a appris à ne jamais supposer qu'une API externe est fiable. La gestion des erreurs, du rate limiting et du cache est aussi importante que le code métier lui-même.
- **Sécurité :** Les principes OWASP sont devenus un réflexe. Je valide désormais systématiquement toutes les entrées utilisateur, et je n'envoie jamais de données sensibles au frontend sans les avoir chiffrées ou filtrées.
- **Gestion de projet solo :** L'organisation avec GitHub Projects et la méthode Agile adaptée m'ont évité de me disperser. Le tableau Kanban avec limite de tâches en cours est devenu mon outil de pilotage quotidien.
- **Documentation :** Écrire la documentation au fur et à mesure (CONTRIBUTING.md, README, commentaires de code) m'a fait gagner du temps quand j'ai dû reprendre une partie du code après plusieurs semaines d'interruption. Sans elle, j'aurais perdu des heures à me rappeler comment démarrer l'environnement.

Chapitre 11
Conclusion et remerciements

11.1 Synthèse du projet

Destiny Raid Companion est une plateforme web destinée aux joueurs de Destiny 2. Elle centralise des guides de raids interactifs, la gestion d'escouades et un calendrier collaboratif. Le projet a été développé en huit mois (environ 60 jours ouvrés) en autonomie complète.

Objectifs atteints

- Authentification OAuth Bungie fonctionnelle et sécurisée (JWT + refresh tokens)
- Guides interactifs pour trois raids majeurs (Vault of Glass, Last Wish, Deep Stone Crypt)
- Création et gestion d'escouades avec système d'invitations
- Architecture 3-tiers respectée : Next.js côté frontend, Node.js/Express côté backend, PostgreSQL pour les données transactionnelles
- Sécurité conforme aux bonnes pratiques OWASP et au RGPD
- Couverture de tests unitaires à 85%

Points d'amélioration identifiés

Les fonctionnalités reportées (calendrier avancé, badges, tests E2E) sont planifiées pour les versions 1.1 et 1.2. Le déploiement continu (CD) reste partiellement manuel, ce qui constitue une dette technique assumée que je compte automatiser dans les prochaines itérations.

Technologies maîtrisées

- **Frontend :** Next.js 14, TypeScript, TailwindCSS
- **Backend :** Node.js, Express.js, Prisma ORM
- **Bases de données :** PostgreSQL, Redis
- **DevOps :** Docker, GitHub Actions, SonarQube
- **Sécurité :** JWT, OAuth 2.0, AES-256, OWASP Top 10

11.2 Perspectives d'évolution

Version 1.1 (juillet 2026)

- Chat intégré pour les escouades (WebSocket)
- Tests E2E complets avec Cypress
- Améliorations UX basées sur les premiers retours utilisateurs
- Version anglaise

Version 1.2 (septembre 2026)

- Système de badges enrichi
- Notifications par email
- Guides pour tous les raids du jeu
- Optimisation du cache Redis

Version 2.0 (2027)

- Application mobile (React Native)
- API publique pour les développeurs tiers
- Intégration Discord (webhooks)
- Analytics avancées

11.3 Remerciements

Je tiens à remercier :

- **Antoine (formateur CDA) :** Pour son accompagnement, ses retours constructifs et la validation de mon dossier.
- **Les testeurs (Thomas, Sarah, Alex) :** Pour leur temps, leurs retours sincères et leur patience.
- **La communauté Destiny 2 :** Pour leur accueil et leur aide dans la compréhension des mécaniques de raids.
- **Bungie :** Pour la mise à disposition de leur API et la richesse de leur univers.
- **La communauté open source :** Pour Next.js, Node.js, Prisma, et tous les outils qui ont rendu ce projet possible.

11.4 Déploiement et documentation

Stratégie de déploiement

Le déploiement repose sur Docker et Docker Compose. J'ai fait ce choix pour garantir que l'application se comporte de la même manière en développement, en test et en production. Les variables sensibles (clé API Bungie) sont injectées via fichier `.env` en local et via GitHub Secrets dans la pipeline CI.

11.4.1 Docker

Le backend est construit avec un Dockerfile multi-stage. Cette approche permet de séparer l'étape de build (où l'on installe toutes les dépendances et on compile) de l'étape d'exécution (où l'on n'embarque que le strict nécessaire). Le résultat est une image finale plus petite et plus sécurisée.

```
# Stage 1: Build
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Production
FROM node:18-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package*.json ./
EXPOSE 3000
ENV NODE_ENV=production
CMD ["node", "dist/index.js"]
```

Le frontend Next.js, quant à lui, est servi par un simple serveur Nginx en production. Le build est effectué au moment de la construction de l'image, ce qui garantit que l'application livrée est optimisée (statique).

```
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/out /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Pour orchestrer l'ensemble (base de données, cache, backend, frontend), j'utilise Docker Compose. La configuration ci-dessous montre comment les services sont interconnectés et comment les health checks permettent à Docker de redémarrer automatiquement un service défaillant.

```
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: raidcompanion
      POSTGRES_USER: user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready␣-U␣user"]
      interval: 10s

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgresql://user:${DB_PASSWORD}@postgres:5432/
          raidcompanion
      REDIS_URL: redis://redis:6379
      BUNGIE_API_KEY: ${BUNGIE_API_KEY}
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - backend

volumes:
```

```
  postgres_data:
```

11.4.2 GitHub (code source)

Le repository est organisé pour séparer clairement les responsabilités. Cette structure facilite la navigation et permet à un nouveau contributeur (ou à moi-même dans six mois) de comprendre rapidement où se trouve chaque partie du code.

```
destiny-raid-companion/
+-- frontend/                # Application Next.js
|   +-- src/
|   +-- public/
|   +-- Dockerfile
+-- backend/                 # API Node.js
|   +-- src/
|   |   +-- controllers/
|   |   +-- services/
|   |   +-- repositories/
|   |   +-- middleware/
|   +-- prisma/
|   +-- Dockerfile
+-- docker-compose.yml
+-- .github/
|   +-- workflows/
|       +-- ci.yml
|       +-- security.yml
+-- README.md
+-- CONTRIBUTING.md
```

URL du dépôt : https://github.com/votre-username/destiny-raid-companion (à remplacer par l'URL réelle)

11.4.3 CI/CD

La pipeline GitHub Actions exécute plusieurs vérifications à chaque push. L'idée est de détecter le plus tôt possible les erreurs, avant même que le code n'atteigne la branche principale.

Les étapes sont les suivantes :

1. Installation des dépendances
2. Lint du code (ESLint)
3. Tests unitaires (Jest)
4. Tests d'intégration (Supertest)
5. Analyse SonarQube
6. Scan de sécurité (npm audit, Snyk)
7. Construction des images Docker (vérification que le build ne casse pas)
8. Déploiement sur staging (branche develop) – actuellement manuel pour la production

11.4.4 SonarQube

SonarQube analyse automatiquement chaque commit. Il me remonte les problèmes de qualité : code dupliqué, complexité excessive, vulnérabilités potentielles. Les métriques actuelles montrent que le code est maintenable et sécurisé.

Métrique	Valeur	Objectif
Couverture de code	85%	> 80%
Duplication	1.2%	< 3%
Complexité cyclomatique	7.3	< 10
Vulnérabilités	0	0

11.4.5 Swagger

La documentation de l'API est générée automatiquement avec Swagger. Elle est accessible à l'URL `/api-docs` une fois l'application démarrée. Elle couvre tous les endpoints principaux :

- `POST /api/auth/callback` – Callback OAuth Bungie
- `GET /api/guides` – Liste des guides
- `GET /api/guides/:id` – Détail d'un guide
- `POST /api/squads` – Création d'une escouade
- `GET /api/squads/:id` – Détail d'une escouade
- `POST /api/squads/:id/invite` – Inviter un membre
- `GET /api/user/profile` – Profil utilisateur
- `GET /api/user/export` – Export RGPD des données

11.5 Liens utiles

- Dépôt GitHub : https://github.com/Lucas-Rochetin/destiny-raid-companion
- Dockerfile reference : https://docs.docker.com/reference/dockerfile/
- GitHub Actions : https://docs.github.com/actions
- SonarQube : https://docs.sonarsource.com/sonarqube/latest/
- Bungie API : https://bungie-net.github.io/